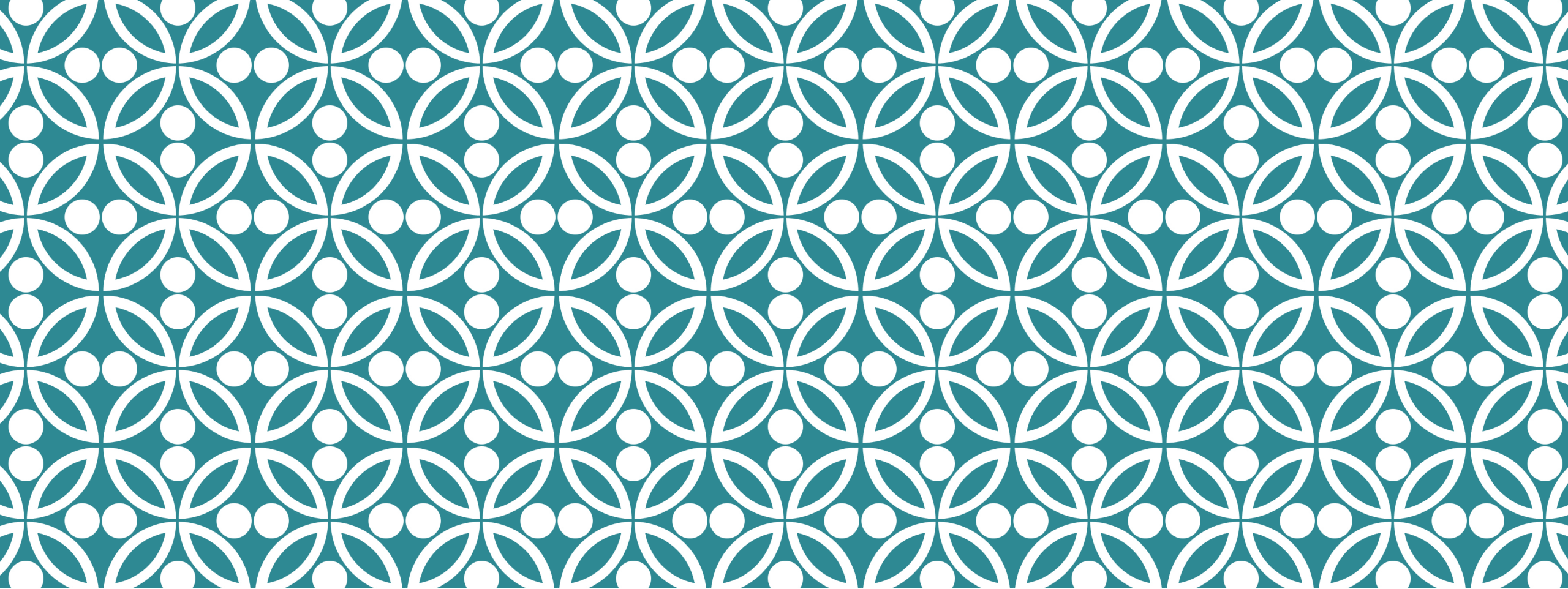




SESSION 2: R-MARKDOWN

DR. SOFIA GIL-CLAVEL

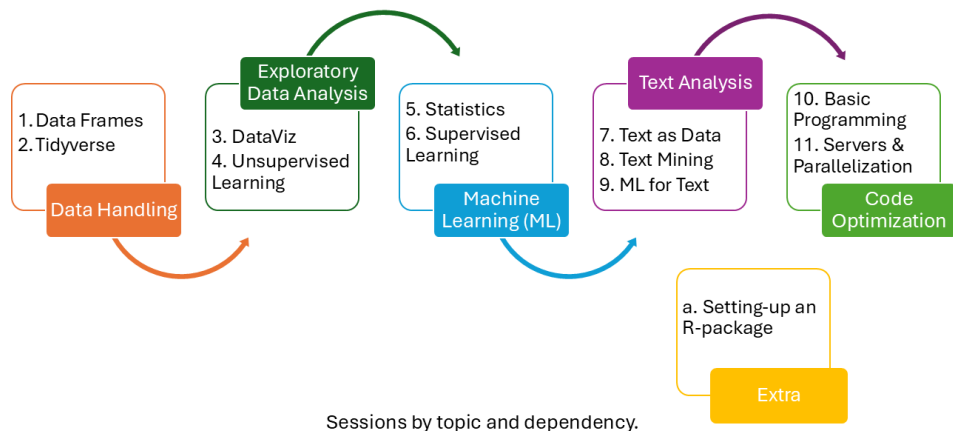
- ❖ Introduction: Creating and rendering documents
- ❖ Adding and formatting text
- ❖ Modifying the settings
- ❖ Building on axioms
- ❖ Tracking students' logic



A COMMERCIAL FROM OUR SPONSOR!

Did you find the first session useful?

Then you should consider following the full course that we offer every year between February and June.



https://societal-analytics.nl/events/20251125_workshop/

To get **all the information about our events**, then **sign up** to receive our newsletter every month!

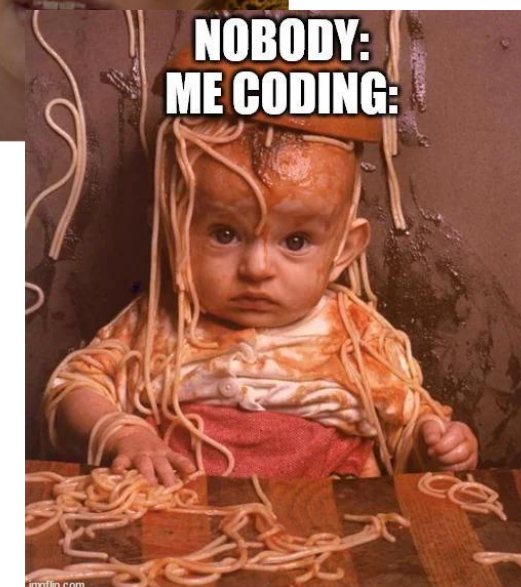
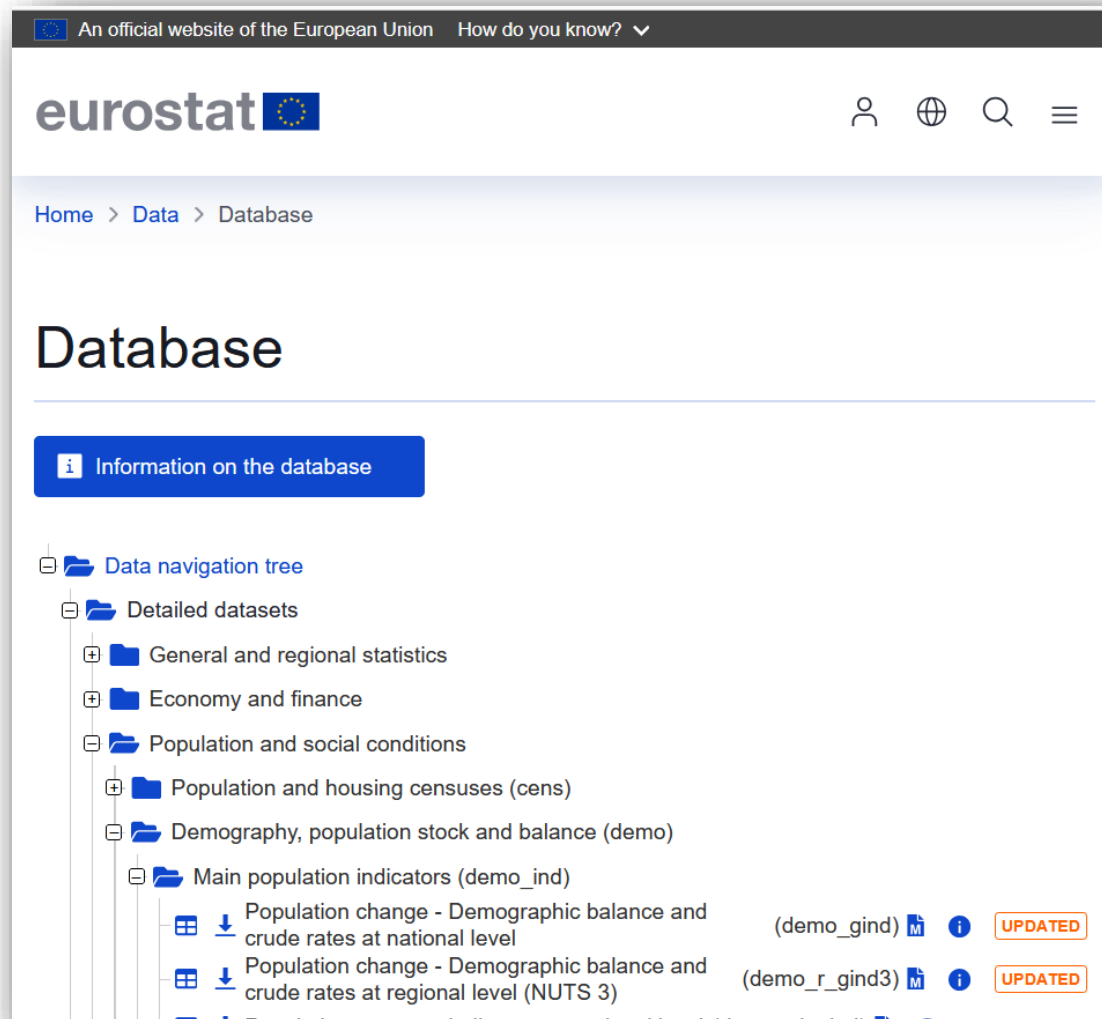


<https://forms.office.com/e/8Bgd2YsasJ>

1. INTRODUCTION: CREATING AND RENDERING DOCUMENTS



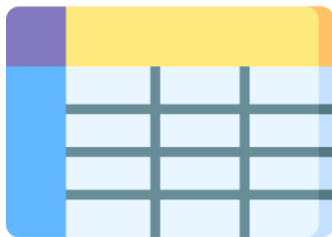
INTRODUCTION

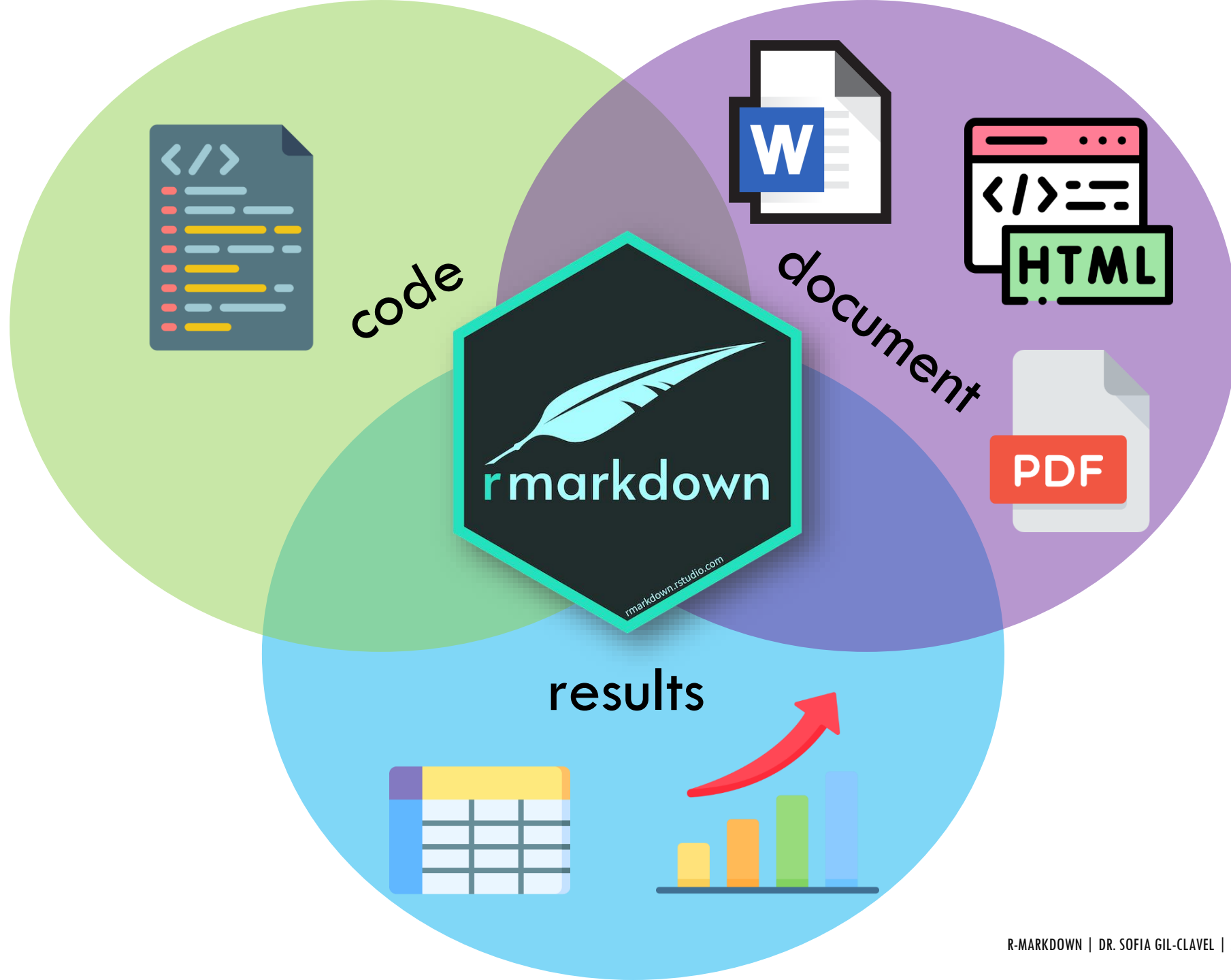


HAS IT HAPPENED TO YOU THAT...?



What if there is a R package that helps you have everything in the same place?







from R Studio

Get Started Gallery Formats Articles Book References

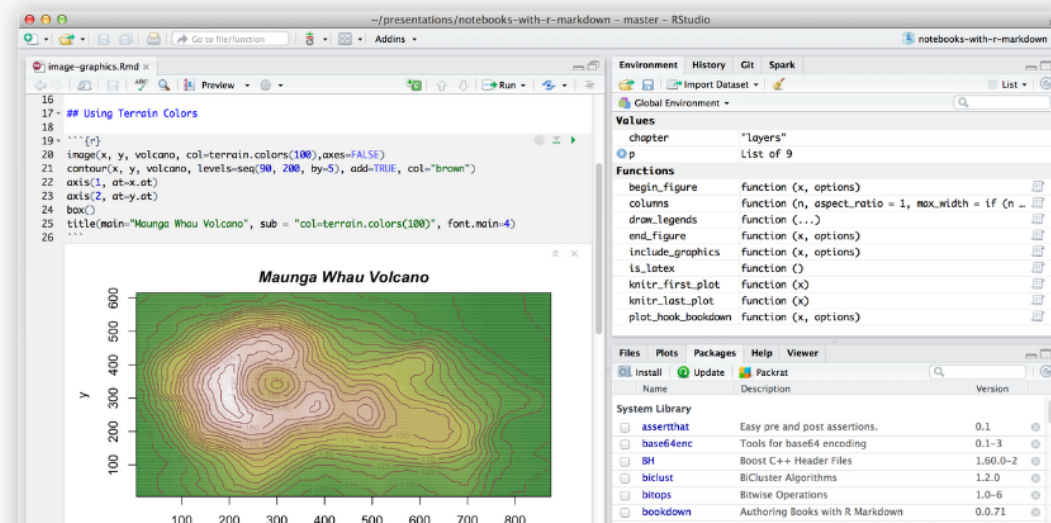
Analyze. Share. Reproduce.

Your data tells a story. Tell it with R Markdown.

Turn your analyses into high quality documents, reports, presentations and dashboards.

R Markdown documents are fully reproducible.

Use a productive [notebook interface](#) to weave together narrative text and code to produce elegantly formatted output. Use [multiple languages](#) including R, Python, and SQL.



BUT FIRST, WE NEED TO INSTALL LATEX

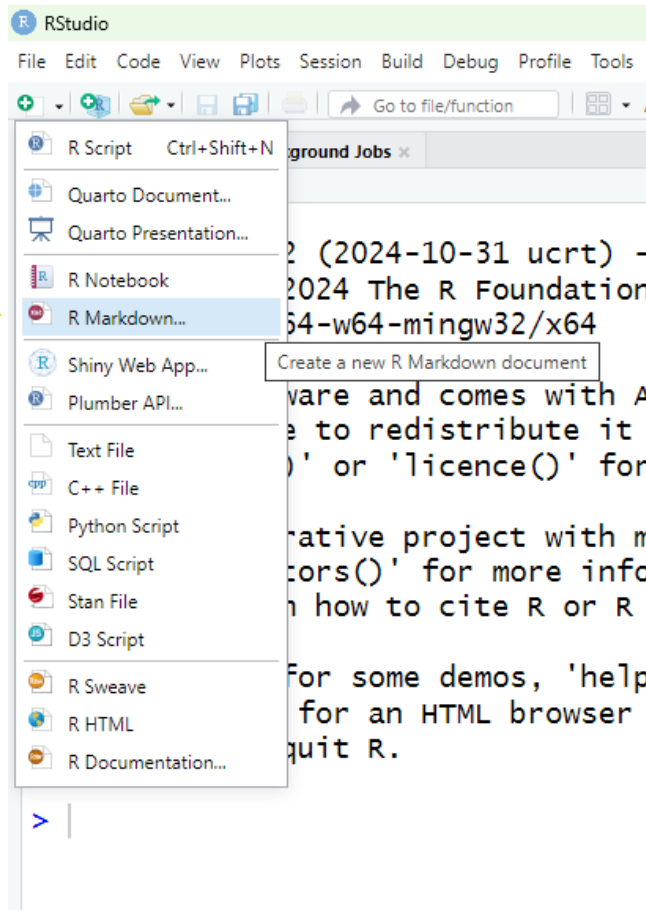
Install the package tinytext.

Then, write and run the following command:

```
tinytex::install_tinytex()
```

It will download and install a small version of a software called LaTeX. This software is kind of like Word, but it is mostly used by mathematicians and programmers because it allows writing math and pseudo code in an easy way. It is also for free.

R-MARKDOWN



New R Markdown

Document
Presentation
Shiny
From Template

Title:

Author:

Date:

☐ Use current date when rendering document

Default Output Format:

☐ HTML
Recommended for... switch to PDF
or Word output a

☒ PDF
PDF output requires TeX (MiKTeX on Windows, MacTeX 2013+ on OS X, TeX Live 2013+ on Linux).

☐ Word
Previewing Word documents requires an installation of MS Word (or Libre/Open Office on Linux).

Create Empty Document

OK Cancel

1. Choose a suitable name

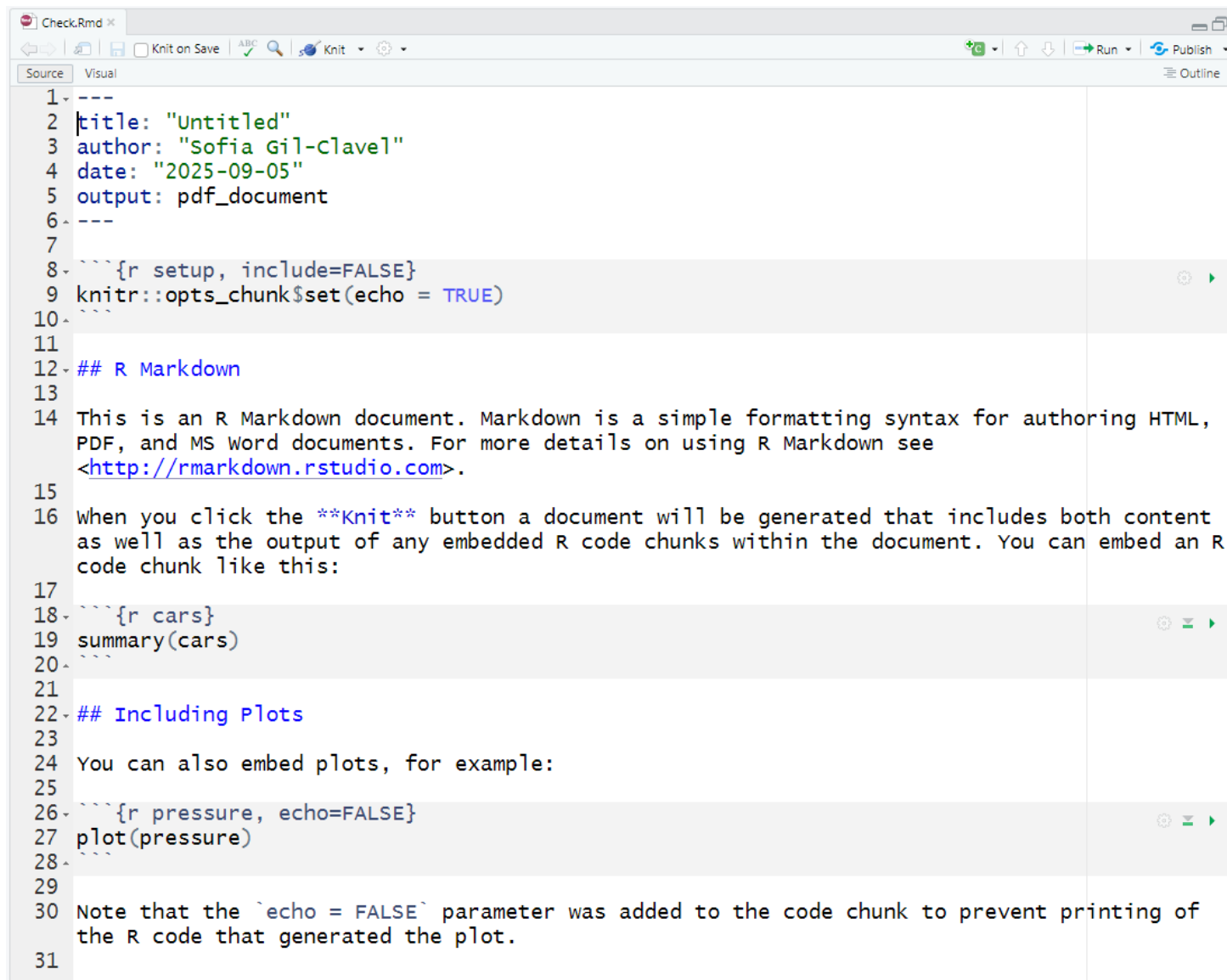
2. Choose the type of output file

3. Apply the changes



THE R-MARKDOWN ELEMENTS

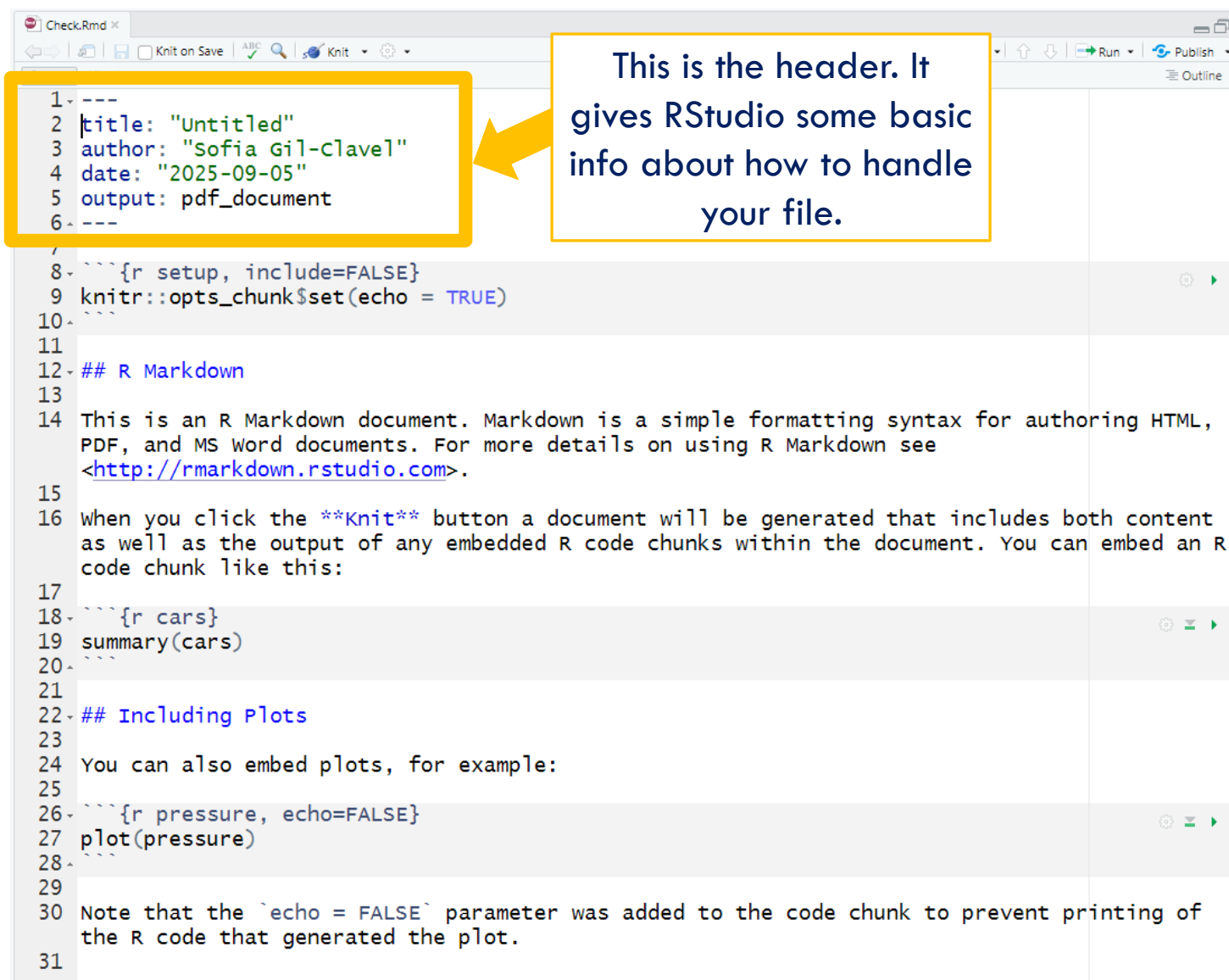
THIS OPENS A SCRIPT THAT LOOKS LIKE:



The image shows a screenshot of a text editor window titled 'Check.Rmd'. The editor has a toolbar at the top with icons for undo, redo, save, and other functions. The main text area contains an R Markdown script. The script starts with a YAML header block followed by a text block, an R code chunk, another text block, another R code chunk, and a final text block. The R code chunks are highlighted in a light gray background. The script is as follows:

```
1 ---
2 title: "Untitled"
3 author: "Sofia Gil-Clavel"
4 date: "2025-09-05"
5 output: pdf_document
6 ---
7
8 ```{r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10 ```
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML,
15 PDF, and MS Word documents. For more details on using R Markdown see
16 <http://rmarkdown.rstudio.com>.
17
18 When you click the Knit button a document will be generated that includes both content
19 as well as the output of any embedded R code chunks within the document. You can embed an R
20 code chunk like this:
21
22 ```{r cars}
23 summary(cars)
24 ```
25
26 ## Including Plots
27
28 You can also embed plots, for example:
29
30 ```{r pressure, echo=FALSE}
31 plot(pressure)
32 ```
33
34 Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of
35 the R code that generated the plot.
```

THIS OPENS A
SCRIPT THAT LOOKS
LIKE:

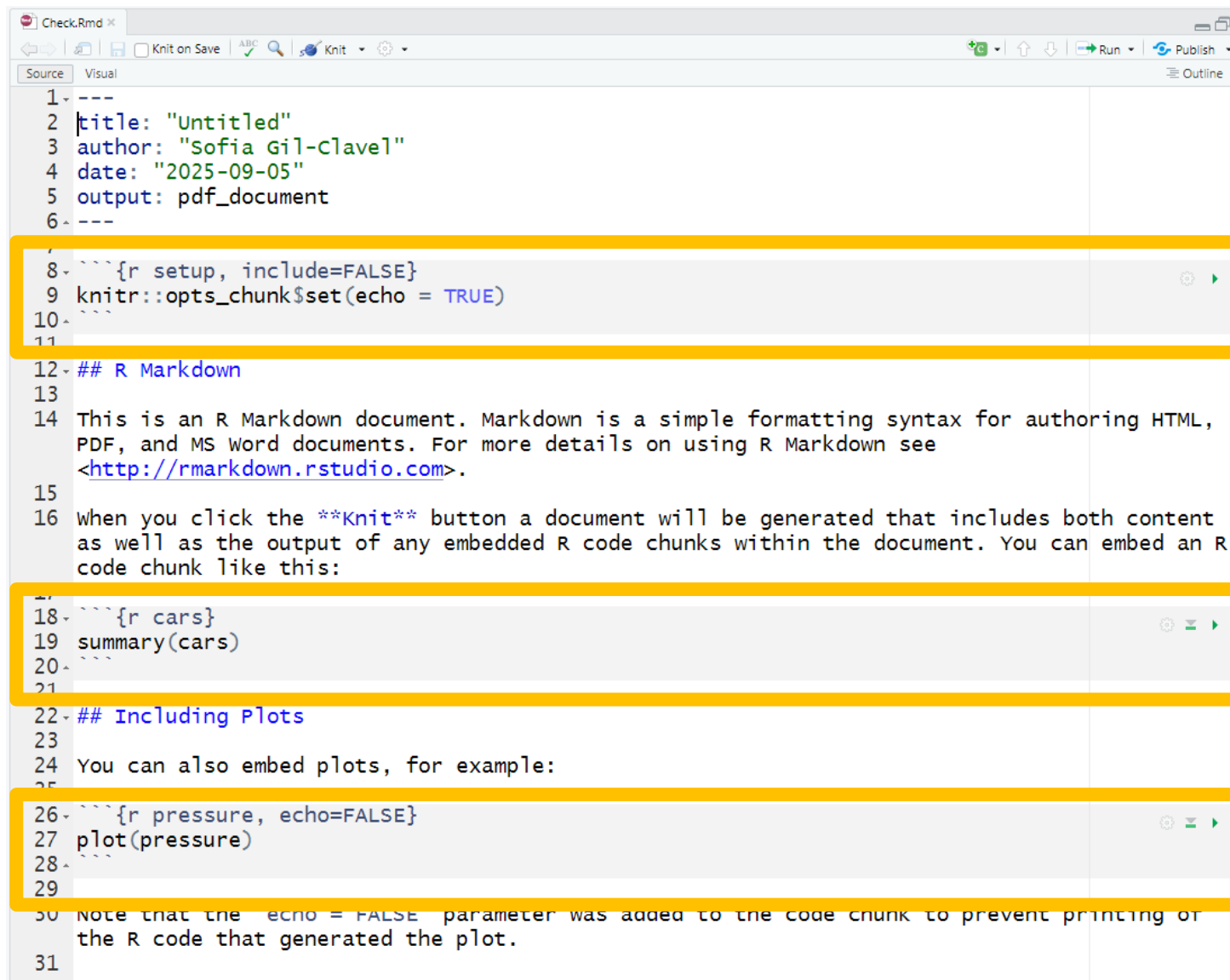


The image shows a screenshot of an R Markdown script in the RStudio editor. The script is titled 'Check.Rmd'. A yellow box highlights the header section (lines 1-6), which contains metadata. A yellow arrow points from this box to a text box on the right that says 'This is the header. It gives RStudio some basic info about how to handle your file.' The script includes several R code chunks with comments and a final note about the 'echo' parameter.

```
1 ---
2 title: "Untitled"
3 author: "Sofia Gil-Clavel"
4 date: "2025-09-05"
5 output: pdf_document
6 ---
7
8 ```{r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10 ```
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML,
15 PDF, and MS Word documents. For more details on using R Markdown see
16 <http://rmarkdown.rstudio.com>.
17
18 When you click the Knit button a document will be generated that includes both content
19 as well as the output of any embedded R code chunks within the document. You can embed an R
20 code chunk like this:
21
22 ```{r cars}
23 summary(cars)
24 ```
25
26 ## Including Plots
27
28 You can also embed plots, for example:
29
30 ```{r pressure, echo=FALSE}
31 plot(pressure)
32 ```
33
34 Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of
35 the R code that generated the plot.
```

THIS OPENS A SCRIPT THAT LOOKS LIKE:

These are called
“Chunks”. In them you
can write code.



```
1 ---
2 title: "Untitled"
3 author: "Sofia Gil-Clavel"
4 date: "2025-09-05"
5 output: pdf_document
6 ---
7
8 {r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML,
15 PDF, and MS Word documents. For more details on using R Markdown see
16 <http://rmarkdown.rstudio.com>.
17
18 When you click the Knit button a document will be generated that includes both content
19 as well as the output of any embedded R code chunks within the document. You can embed an R
20 code chunk like this:
21
22 {r cars}
23 summary(cars)
24
25
26 ## Including Plots
27
28 You can also embed plots, for example:
29
30 {r pressure, echo=FALSE}
31 plot(pressure)
```

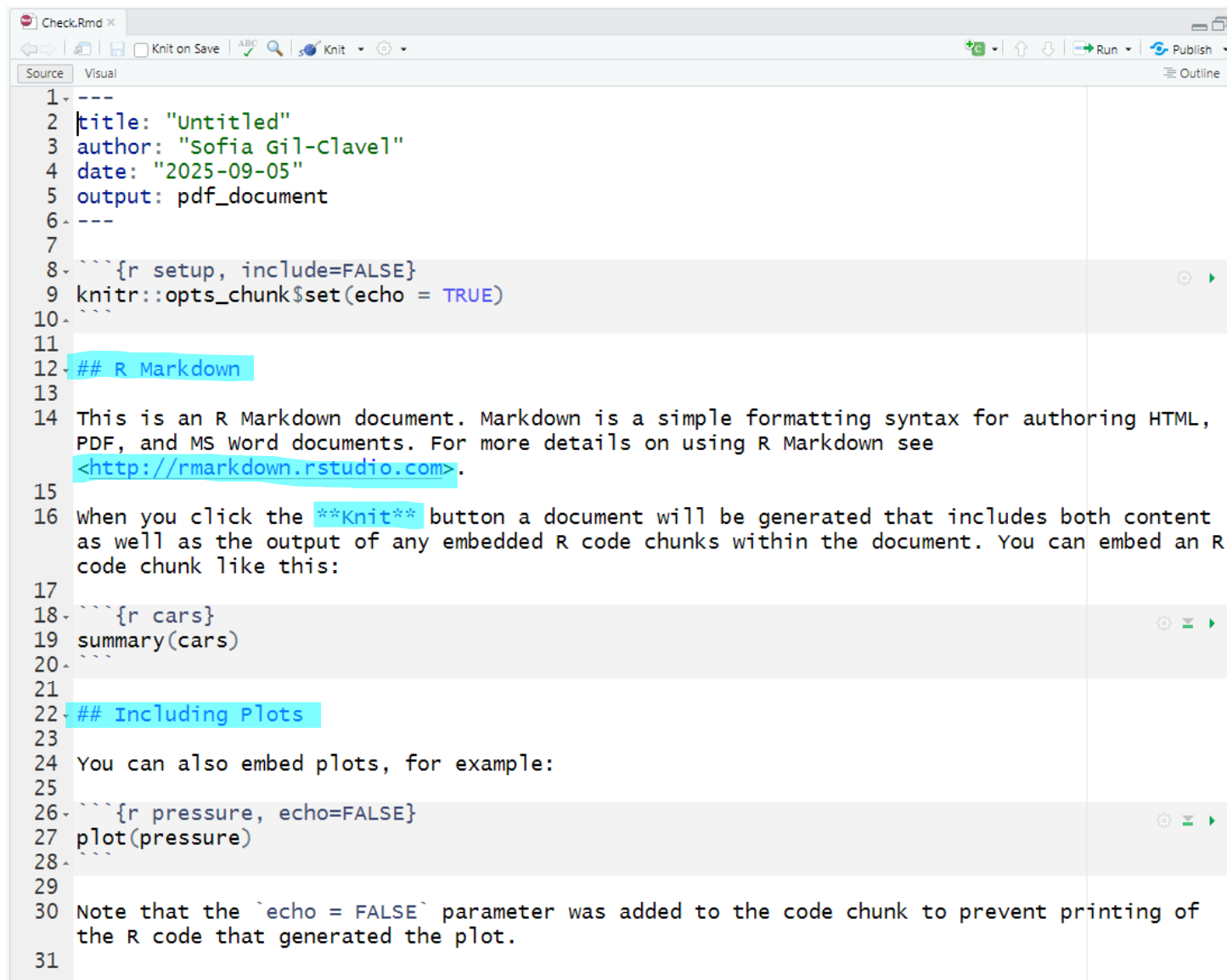
THIS OPENS A SCRIPT THAT LOOKS LIKE:

You can write normal text outside the chunks.

```
1 ---
2 |title: "Untitled"
3 |author: "Sofia Gil-Clavel"
4 |date: "2025-09-05"
5 |output: pdf_document
6 ---
7
8 ```{r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10 ```
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML,
15 PDF, and MS Word documents. For more details on using R Markdown see
16 <http://rmarkdown.rstudio.com>.
17
18 When you click the Knit button a document will be generated that includes both content
19 as well as the output of any embedded R code chunks within the document. You can embed an R
20 code chunk like this:
21
22 ```{r cars}
23 summary(cars)
24 ```
25
26 ## Including Plots
27
28 You can also embed plots, for example:
29
30 ```{r pressure, echo=FALSE}
31 plot(pressure)
32 ```
33
34 Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of
35 the R code that generated the plot.
```

THIS OPENS A SCRIPT THAT LOOKS LIKE:

This is **special syntax** that changes how the text looks in the final document.

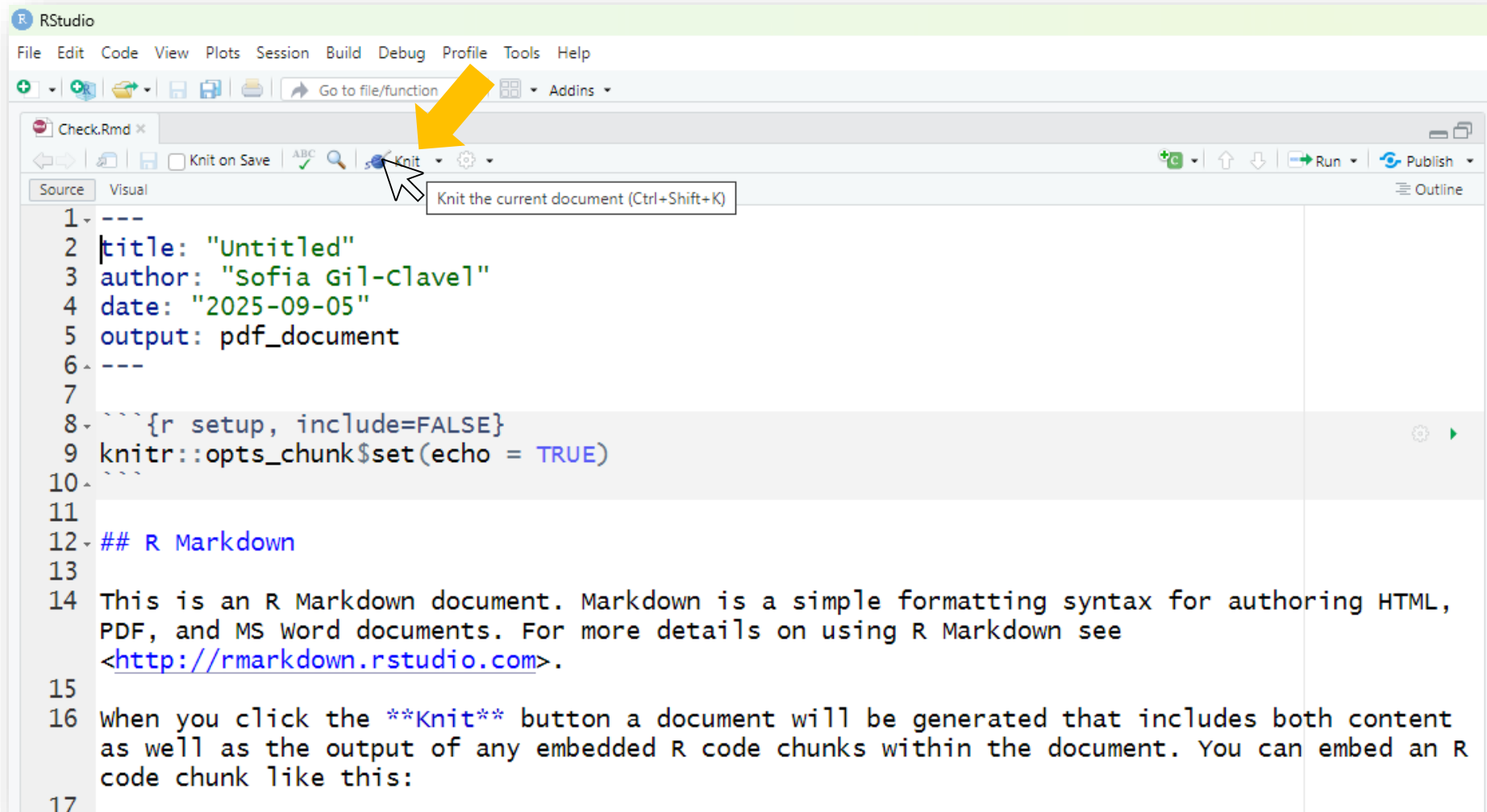


```
1 ---
2 |title: "Untitled"
3 |author: "Sofia Gil-Clavel"
4 |date: "2025-09-05"
5 |output: pdf_document
6 ---
7
8 ```{r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE)
10 ```
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML,
15 PDF, and MS Word documents. For more details on using R Markdown see
16 <http://rmarkdown.rstudio.com>.
17
18 When you click the Knit button a document will be generated that includes both content
19 as well as the output of any embedded R code chunks within the document. You can embed an R
20 code chunk like this:
21
22 ```{r cars}
23 summary(cars)
24 ```
25
26 ## Including Plots
27
28 You can also embed plots, for example:
29
30 ```{r pressure, echo=FALSE}
31 plot(pressure)
32 ```
33
34 Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of
35 the R code that generated the plot.
```



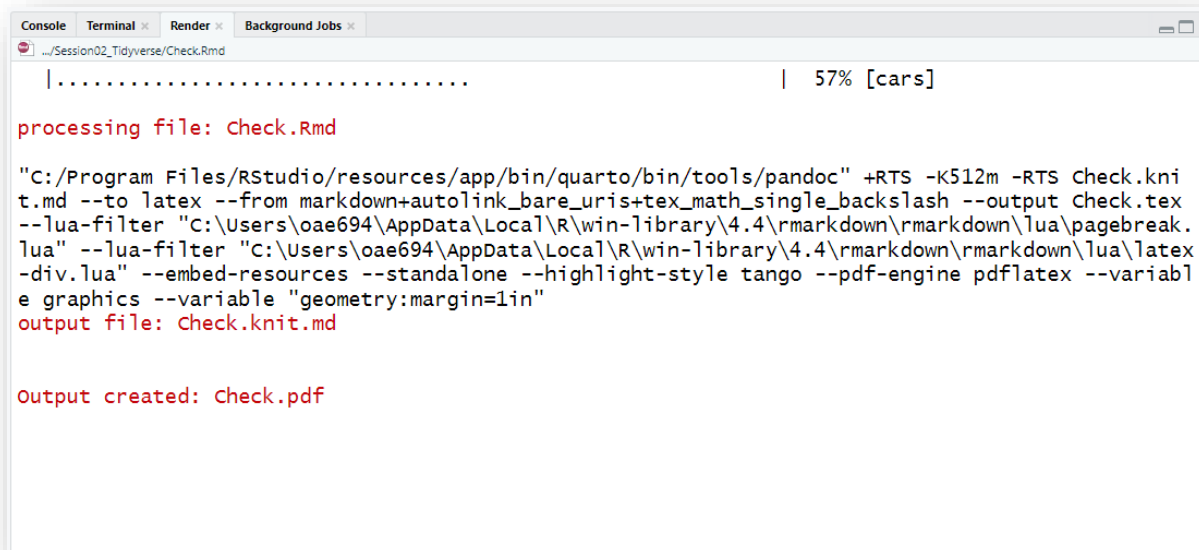
FROM R-MARKDOWN TO PDF, WORD, OR HTML.

LET'S RENDER YOUR R-MARKDOWN FILE



IF EVERYTHING GOES WELL THEN...

1 The panel “Render” will display something similar to:



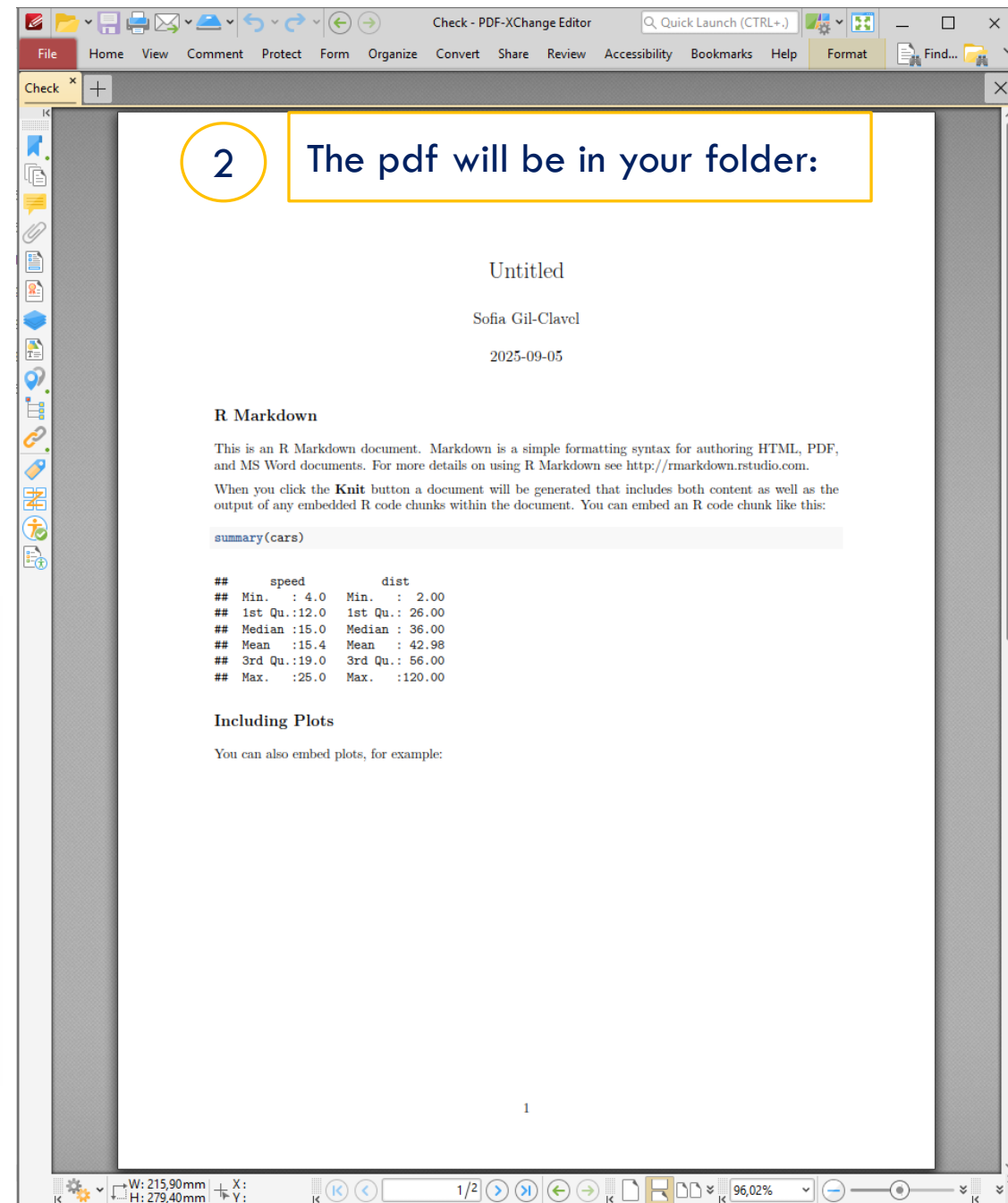
```
Console Terminal Render Background Jobs
~/Session02_Tidyverse/Check.Rmd

|.....| 57% [cars]

processing file: Check.Rmd

"C:/Program Files/RStudio/resources/app/bin/quarto/bin/tools/pandoc" +RTS -k512m -RTS Check.kn
t.md --to latex --from markdown+autolink_bare_uris+tex_math_single_backslash --output Check.tex
--lua-filter "C:\Users\oae694\AppData\Local\R\win-library\4.4\rmarkdown\rmarkdown\lua\pagebreak.
lua" --lua-filter "C:\Users\oae694\AppData\Local\R\win-library\4.4\rmarkdown\rmarkdown\lua\latex
-div.lua" --embed-resources --standalone --highlight-style tango --pdf-engine pdflatex --variabl
e graphics --variable "geometry:margin=1in"
output file: Check.knit.md

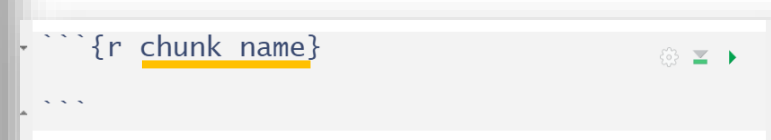
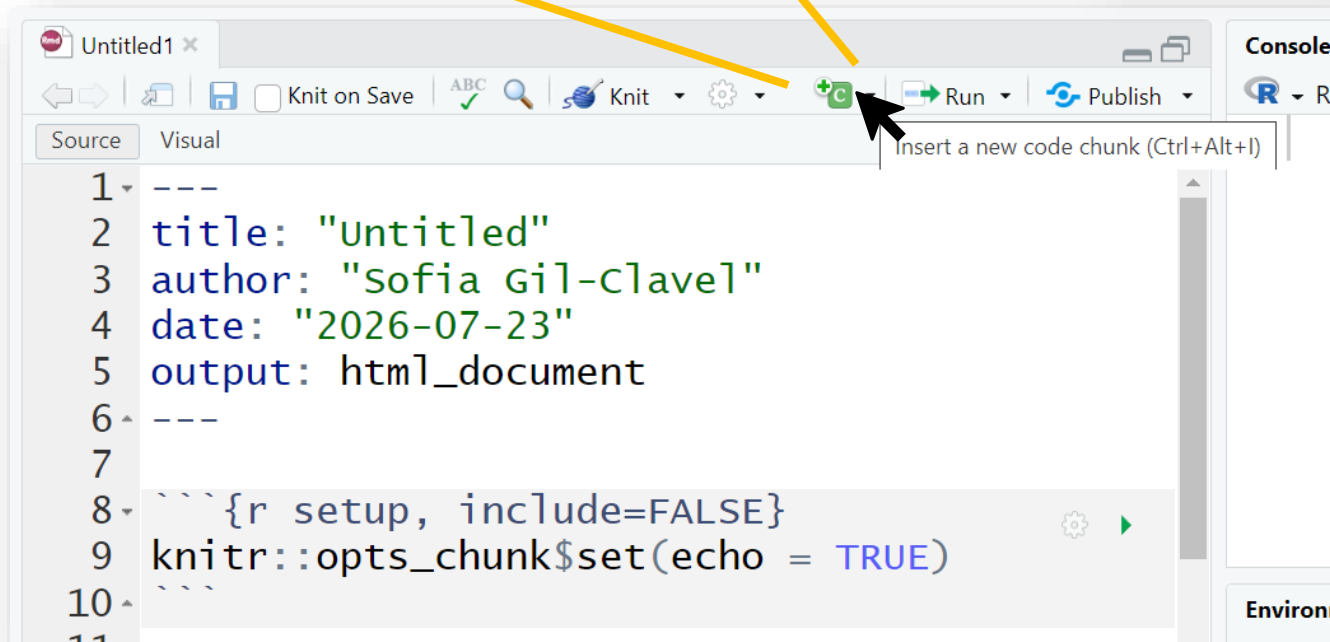
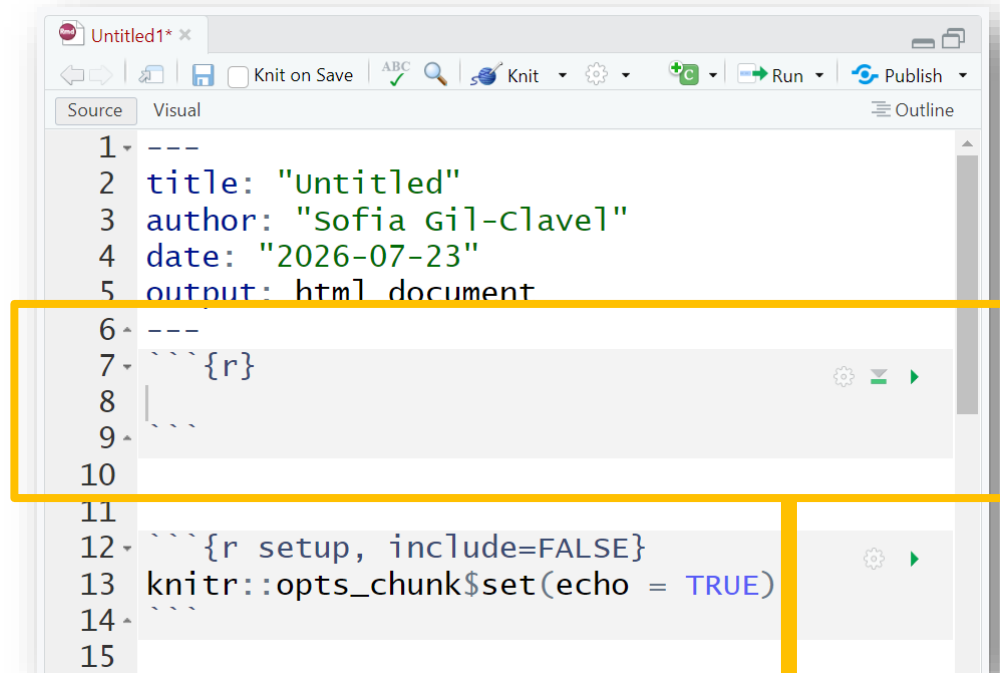
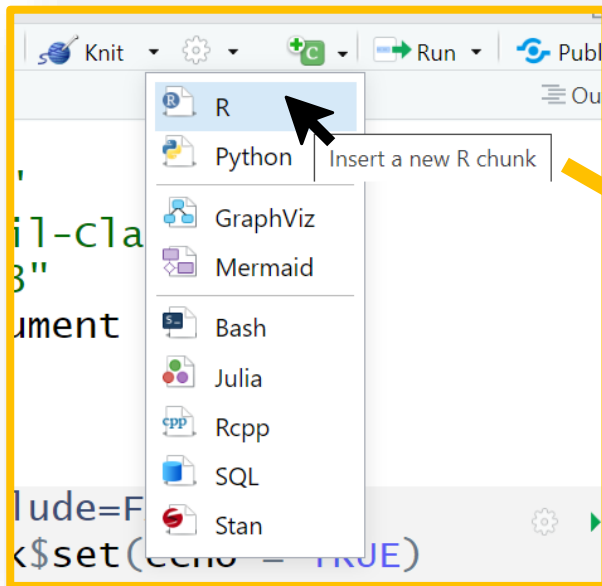
Output created: Check.pdf
```





THE CODE GOES IN CHUNKS!

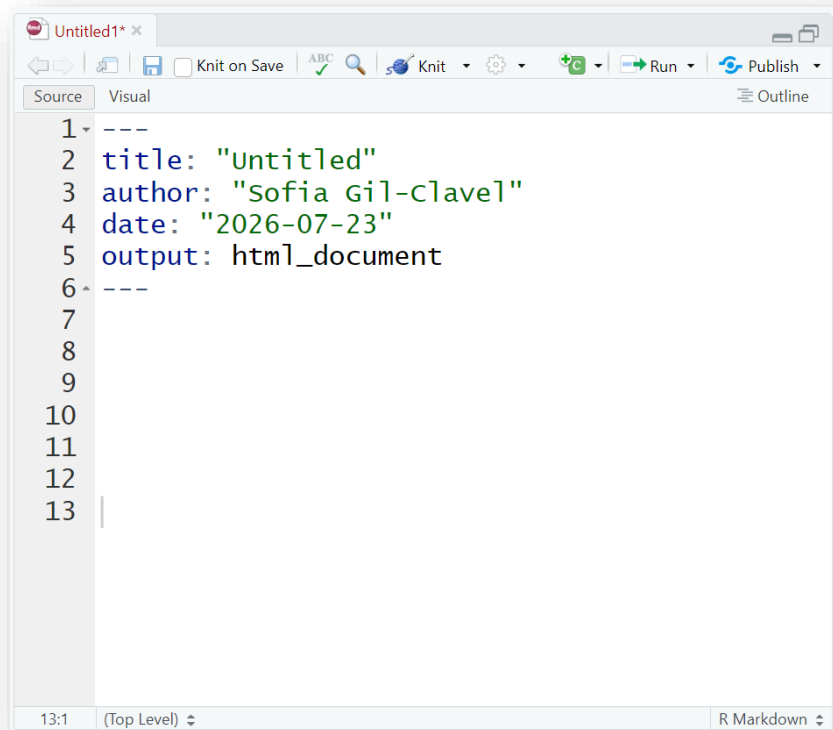
THE CODE GOES IN CHUNKS!



Chunk names
must be unique.

EXERCISE 1.1

Delete all the elements in the R-Markdown, except for the header, i.e. lines 1 to 6.



```
1 ---
2 title: "Untitled"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-23"
5 output: html_document
6 ---
7
8
9
10
11
12
13
```

Create a new chunk in which you clean the environment.

Hint: Use the functions `rm(list=ls())` and `gc()`.

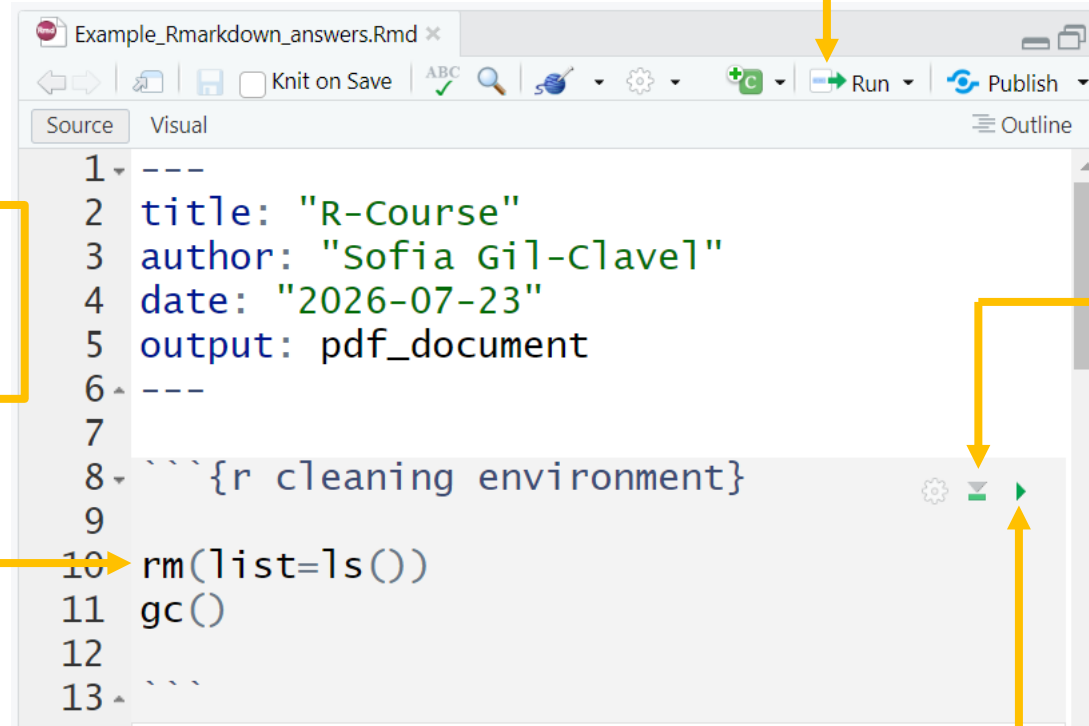
Remember to give your chunk a name.

Let me know when you are done.

RUNNING CODE

Option 2: Whole chunk.
Press the “Run” button at the top of the Script panel.

Option 1: By line.
Place yourself the line and press the keys “Ctrl” + “Enter”.



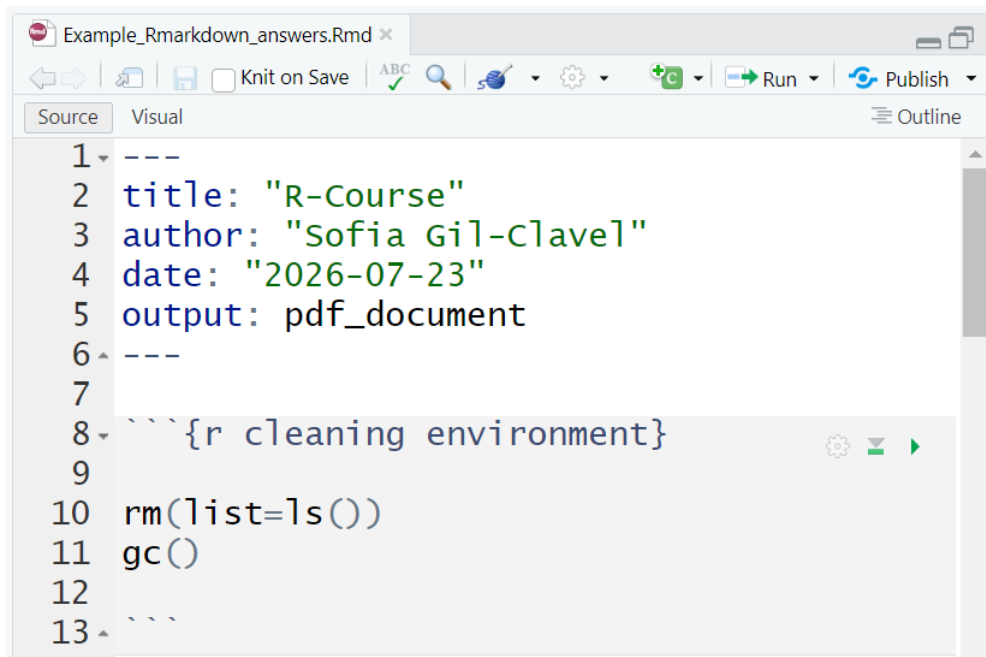
```
1 ---
2 title: "R-Course"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-23"
5 output: pdf_document
6 ---
7
8 ```{r cleaning environment}
9
10 rm(list=ls())
11 gc()
12
13 ```
```

Option 4: All previous chunks.
Press the “Downward” play button.

Option 3: Whole chunk.
Press the green “play” button at the top of the chunk.

EXERCISE 1.2

Let's continue working on your R-Markdown.



```
1 ---
2 title: "R-Course"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-23"
5 output: pdf_document
6 ---
7
8 ```{r cleaning environment}
9
10 rm(list=ls())
11 gc()
12
13 ```
```

For each of the following steps create a new chunk with a suitable name:

1. Load the tidyverse package.
2. Turn the “iris” database into a tibble.
3. Obtain the average of Petal.Length by Species.
Hint: Use the functions `group_by()` and `summarise()`.
4. Make a bar plot of the results.

EXERCISE 1.3

1. Go to the final chunk.
2. Run all the previous chunks.
3. Run the final chunk.
4. Render your file!

5 MINS BREAK



2. ADDING AND FORMATTING TEXT



ADDING TEXT

WHITE SECTIONS ARE FOR TEXT

```
1 ---
2 title: "R-Course"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-23"
5 output: pdf_document
6 ---
7 
8 {r cleaning environment}
9 
10 rm(list=ls())
11 gc()
12 
13 
14 
15 
16 {r loading packages}
17 
18 library(tidyverse)
19 
20 
21 
22 
23 {r opening data}
24 
25 IRIS<-tibble(iris)
26 
27 
28
```

```
1 ---
2 title: "R-Course"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-23"
5 output: pdf_document
6 ---
7 
8 In the following chunk, I clean the R
environment.
9 
10 {r cleaning environment}
11 
12 rm(list=ls())
13 gc()
14 
15 
16 
17 Then I load the packages.
18 
19 {r loading packages}
20 
21 library(tidyverse)
22 
23 
24 
25 Then I turn the database iris into a tibble.
26 
27 {r opening data}
28 
29 IRIS<-tibble(iris)
30
```

EXERCISE 2.1

1. Write as plain text the description of what you did in each chunk of the R-Markdown.
2. Render the file.



R-MARKDOWN SYNTAX

R-MARKDOWN SYNTAX

You can add headers, use different font styles, create lists, and more!

You just need to use some embedded commands, then R-Markdown will translate them in the final document.

```
1 ---
2 title: "R-Course"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-23"
5 output: pdf_document
6 ---
7
8 # Example code for R-Markdown usage
9
10 ## Cleaning the environment
11
12 In the following chunk, I clean the *R environment*.
13 ```{r cleaning environment}
14
15 rm(list=ls())
16 gc()
17
18 ```
19
20
21 ## Loading the packages
22
23 Then I load the **packages**.
24 ```{r loading packages}
25
26 library(tidyverse)
27
28 ```
29
```

R-Course

Sofia Gil-Clavel

2026-07-23

Example code for R-Markdown usage

Cleaning the environment

In the following chunk, I clean the R environment.

```
rm(list=ls())
gc()
```

##	used (Mb)	gc trigger (Mb)	max used (Mb)
## Ncells	552613	29.6	1235665
## Vcells	1051176	8.1	8388608

Loading the packages

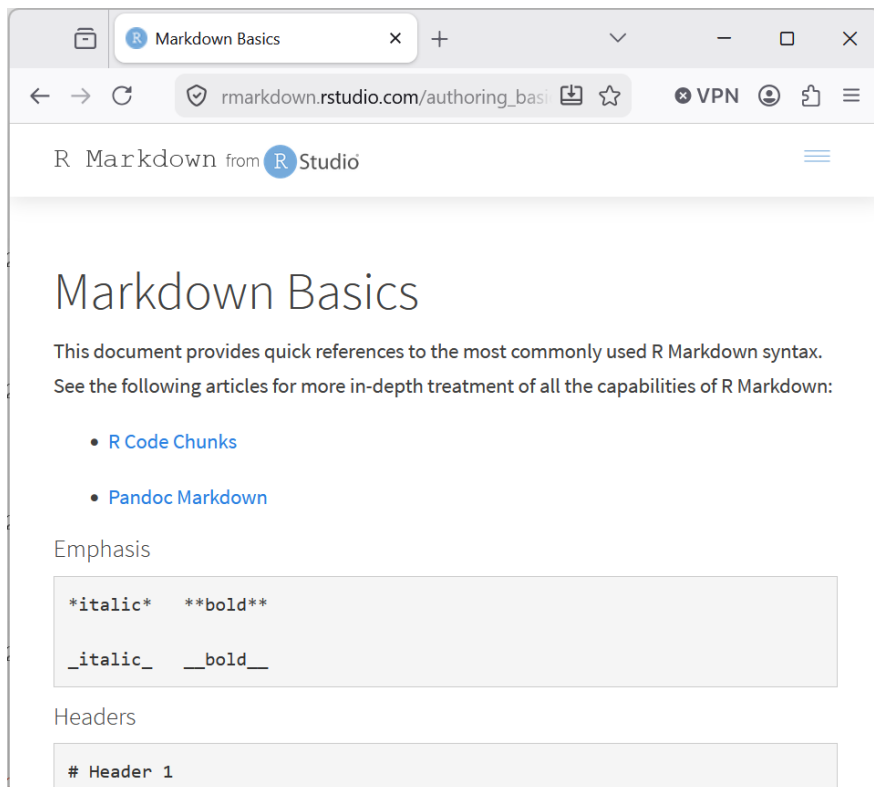
Then I load the packages.

```
library(tidyverse)
```

R-MARKDOWN SYNTAX

You can find all the basic commands here:

https://rmarkdown.rstudio.com/authoring_basics.html



R Markdown Basics

This document provides quick references to the most commonly used R Markdown syntax. See the following articles for more in-depth treatment of all the capabilities of R Markdown:

- [R Code Chunks](#)
- [Pandoc Markdown](#)

Emphasis

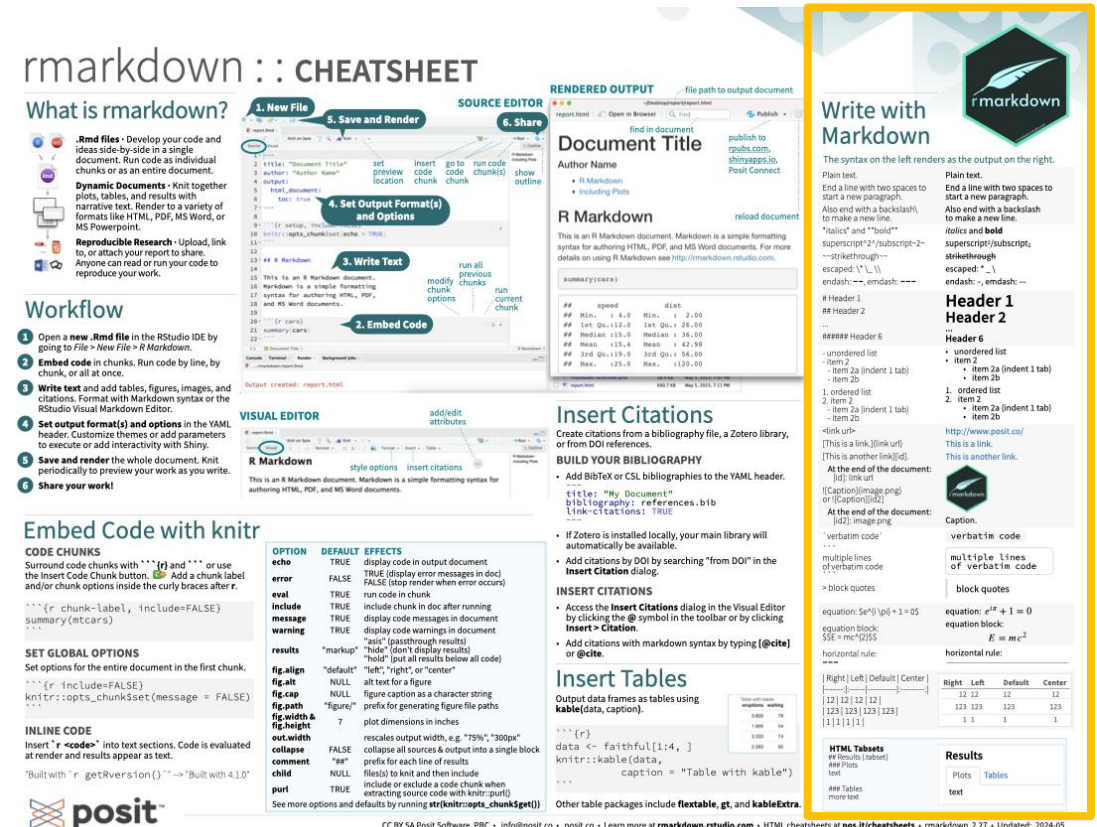
italic ****bold****

`_italic_` `__bold__`

Headers

Header 1

You can also find them in the R-Markdown cheat sheet.



rmarkdown::CHEATSHEET

What is rmarkdown?

Workflow

Embed Code with knitr

Insert Citations

Insert Tables

EXERCISE 2.2

Using the R-Markdown syntax from Posit or in the Cheat Sheet add the following elements to your final file:

1. Title
2. The following sections:
 - Preparing the environment
 - Data Handling
 - Data Visualization
3. Three words in italics.
4. Three words in bold.
5. Under the header add the list with the names of the sections in your file.

3. MODIFYING THE SETTINGS

R-Course

Sofia Gil-Clavel

2026-07-23

Example code for R-Markdown usage

Preparing the environment

In the following chunk, I clean the *R* environment.

```
rm(list=ls())
gc()
```

```
##           used (Mb) gc trigger (Mb) max used (Mb)
## Ncells  552618 29.6   1235680   66   686457 36.7
## Vcells 1051244  8.1    8388608   64   1877227 14.4
```

Then I load the packages.

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.3
## Warning: package 'ggplot2' was built under R version 4.4.3
## Warning: package 'tibble' was built under R version 4.4.3
## Warning: package 'purrr' was built under R version 4.4.3

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble     3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Performing Data Handling

For the *data handling*, first, I turn the database iris into a **tibble**.

1

Sometimes it is not necessary to show all outputs.

```
IRIS<-tibble(iris)
```

Finally, I obtain the **mean** petal length by iris species.

```
IRIS2<-IRIS%>%
  group_by(Species)%>%
  summarise(MEAN=mean(Petal.Length))
```

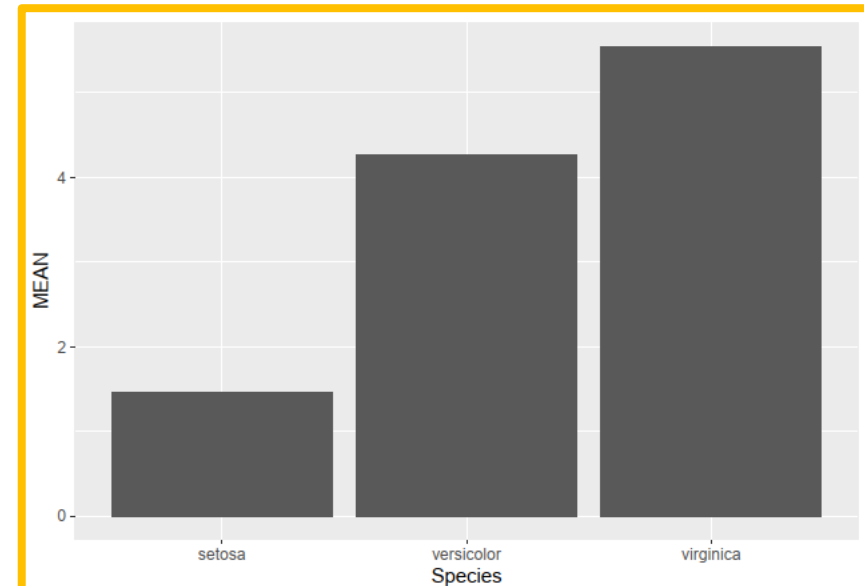
```
IRIS2
```

```
## # A tibble: 3 x 2
##   Species      MEAN
##   <fct>      <dbl>
## 1 setosa      1.46
## 2 versicolor  4.26
## 3 virginica   5.55
```

Performing Data Visualization

To visualize the *results*, I plot the values using bar plots.

```
IRIS2%>%
  ggplot(aes(Species, MEAN))+
  geom_col()
```



Other times, it is necessary to give better format to the results.

2



CHANGING CHUNK PARAMETERS

Chunks are like any other function. They have parameters and based on these parameters the format of the code's output is modified.

Add a coma after the chunk name and between parameters.

```
```${r cleaning environment, echo=FALSE}  
rm(list=ls())
gc()
```
```

This is the parameter “echo” with the assigned value of “FALSE”.

Example code for R-Markdown usage

Preparing the environment

In the following chunk, I clean the *R* environment.

```
rm(list=ls())  
gc()
```

```
##          used (Mb) gc trigger (Mb) max used (Mb)  
## Ncells  552618 29.6   1235680   66   686457 36.7  
## Vcells 1051244  8.1   8388608   64  1877227 14.4
```

Then I load the packages.

```
library(tidyverse)
```

After echo=FALSE

Example code for R-Markdown usage

Preparing the environment

In the following chunk, I clean the *R* environment.

```
##          used (Mb) gc trigger (Mb) max used (Mb)  
## Ncells  552621 29.6   1235688   66   686457 36.7  
## Vcells 1051266  8.1   8388608   64  1877227 14.4
```

Then I load the packages.

```
library(tidyverse)
```

You can find the list of chunk parameters and their description here:

<https://pkg.yihui.org/rmarkdown-book/r-code>

There are a large number of chunk options in **knitr** documented at <https://yihui.name/knitr/options>. We list a subset of them below:

- `eval` : Whether to evaluate a code chunk.
- `echo` : Whether to echo the source code in the output document (someone may not prefer reading your smart source code but only results).
- `results` : When set to `'hide'`, text output will be hidden; when set to `'asis'`, text output is written “as-is,” e.g., you can write out raw Markdown text from R code (like `cat('**Markdown** is cool.\n')`). By default, text output will be wrapped in verbatim elements (typically plain code blocks).
- `warning`, `message`, and `error` : Whether to show warnings, messages, and errors in the output document. Note that if you set `error = FALSE`, `rmarkdown::render()` will halt on error in a code chunk, and the error will be displayed in the R console. Similarly, when `warning = FALSE` or `message = FALSE`, these messages will be shown in the R console.
- `include` : Whether to include anything from a code chunk in the output document. When `include = FALSE`, this whole code chunk is excluded in the output, but note that it will still be evaluated if `eval = TRUE`. When you are trying to set `echo = FALSE`, `results = 'hide'`, `warning = FALSE`, and `message = FALSE`, chances are you simply mean a single option `include = FALSE` instead of suppressing different types of text output individually.



rmarkdown :: CHEAT SHEET

What is rmarkdown?



.Rmd files - Develop your code and ideas side-by-side in a single document. Run code as individual chunks or as an entire document.



Dynamic Documents - Knit together plots, tables, and results with narrative text. Render to a variety of formats like HTML, PDF, MS Word, or MS Powerpoint.



Reproducible Research - Upload, link to, or attach your report to share. Anyone can read or run your code to reproduce your work.

Workflow

1. Open a new **.Rmd** file in the RStudio IDE by going to **File > New File > R Markdown**.
2. **Embed code** in chunks. Run code by line, by chunk, or all at once.
3. **Write text** and add tables, figures, images, and citations. Format with Markdown syntax or the RStudio Visual Markdown Editor.
4. **Set output format(s) and options** in the YAML header. Customize themes or add parameters to execute or add interactivity with Shiny.
5. **Save and render** the whole document. Knit periodically to preview your work as you write.
6. **Share your work!**

Embed Code with knitr

CODE CHUNKS

Surround code chunks with ````{r}```` or use the **Insert Code Chunk** button. Add a chunk label and/or chunk options inside the curly braces after `r`.

```
```{r chunk-label, include=FALSE}
summary(mtcars)
```
```

SET GLOBAL OPTIONS

Set options for the entire document in the first chunk.

```
```{r include=FALSE}
knitr::opts_chunk$set(message = FALSE)
```
```

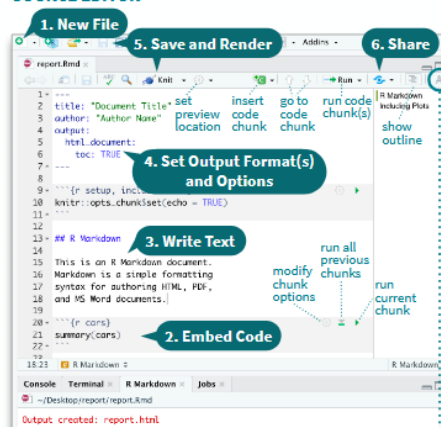
INLINE CODE

Insert ``r <code>`` into text sections. Code is evaluated at render and results appear as text.

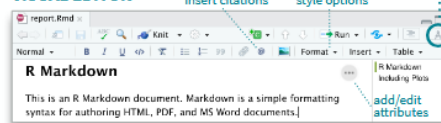
```
"Built with `r getRversion()`" -> "Built with 4.1.0"
```



SOURCE EDITOR



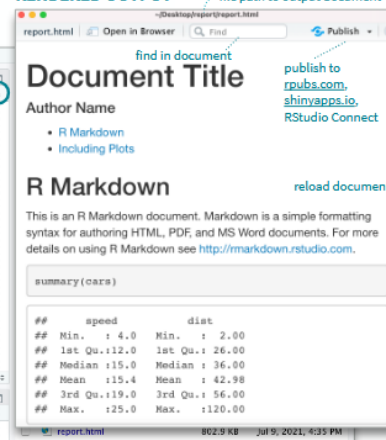
VISUAL EDITOR



| OPTION | DEFAULT | EFFECTS |
|-----------------------------------|-----------|---|
| echo | TRUE | display code in output document |
| error | FALSE | TRUE (display error messages in doc) FALSE (stop render when error occurs) |
| eval | TRUE | run code in chunk |
| include | TRUE | include chunk in doc after running |
| message | TRUE | display code messages in document |
| warning | TRUE | display code warnings in document |
| results | "markup" | "asis" (passthrough results)
"hide" (don't display results)
"hold" (put all results below all code) |
| fig.align | "default" | "left", "right", or "center" |
| fig.alt | NULL | alt text for a figure |
| fig.cap | NULL | figure caption as a character string |
| fig.path | "figure/" | prefix for generating figure file paths |
| fig.width & fig.height | 7 | plot dimensions in inches |
| out.width | NULL | rescales output width, e.g. "75%", "300px" |
| collapse | FALSE | collapse all sources & output into a single block |
| comment | "##" | prefix for each line of results |
| child | NULL | files(s) to knit and then include |
| pur1 | TRUE | include or exclude a code chunk when extracting source code with <code>knitr::pur1()</code> |

See more options and defaults by running `str(knitr::opts_chunk$get())`

RENDERED OUTPUT



Insert Citations

Create citations from a bibliography file, a Zotero library, or from DOI references.

BUILD YOUR BIBLIOGRAPHY

- Add BibTeX or CSL bibliographies to the YAML header.

```
title: "My Document"
bibliography: references.bib
link-citations: TRUE
```

If Zotero is installed locally, your main library will automatically be available.

Add citations by DOI by searching "from DOI" in the **Insert Citation** dialog.

INSERT CITATIONS

Access the **Insert Citations** dialog in the Visual Editor by clicking the @ symbol in the toolbar or by clicking **Insert > Citation**.

Add citations with markdown syntax by typing `[@cite]` or `@cite`.

Insert Tables

Output data frames as tables using `table()` (data, caption).

```
```{r}
data <- faithful[1:4,]
knitr::kable(data,
 caption = "Table with kable")
```
```

Other table packages include `flextable`, `gt`, and `kableExtra`.

Write with Markdown

The syntax on the left renders as the output on the right.

Plain text.

End a line with two spaces to start a new paragraph.
Also end with a backslash to make a new line.

`"italics"` and `"bold"`
`superscript^2/subscript=2`
`~strikethrough~`
escaped: `" \ _ | \`
endash: `--`, emdash: `---`

Plain text.

End a line with two spaces to start a new paragraph.
Also end with a backslash to make a new line.

italics and **bold**
^{superscript}/_{subscript}
~~strikethrough~~
escaped: " \ _ | \
endash: --, emdash: ---

Header 1

Header 2

Header 3

Header 4

Header 5

Header 6

Header 7

Header 8

Header 9

Header 10

Header 11

Header 12

Header 13

Header 14

Header 15

Header 16

Header 17

Header 18

Header 19

Header 20

Header 21

Header 22

Header 23

Header 24

Header 25

Header 26

Header 27

Header 28

Header 29

Header 30

Header 31

Header 32

Header 33

Header 34

Header 35

Header 36

Header 37

Header 38

Header 39

Header 40

Header 41

Header 42

Header 43

Header 44

Header 45

Header 46

Header 47

Header 48

Header 49

Header 50

Header 51

Header 52

Header 53

Header 54

Header 55

Header 56

Header 57

Header 58

Header 59

Header 60

Header 61

Header 62

Header 63

Header 64

Header 65

Header 66

Header 67

Header 68

Header 69

Header 70

Header 71

Header 72

Header 73

Header 74

Header 75

Header 76

Header 77

Header 78

Header 79

Header 80

Header 81

Header 82

Header 83

Header 84

Header 85

Header 86

Header 87

Header 88

Header 89

Header 90

Header 91

Header 92

Header 93

Header 94

Header 95

Header 96

Header 97

Header 98

Header 99

Header 100

Header 101

Header 102

Header 103

Header 104

Header 105

Header 106

Header 107

Header 108

Header 109

Header 110

Header 111

Header 112

Header 113

Header 114

Header 115

Header 116

Header 117

Header 118

Header 119

Header 120

Header 121

Header 122

Header 123

Header 124

Header 125

Header 126

Header 127

Header 128

Header 129

Header 130

Header 131

Header 132

Header 133

Header 134

Header 135

Header 136

Header 137

Header 138

Header 139

Header 140

Header 141

Header 142

Header 143

Header 144

Header 145

Header 146

Header 147

Header 148

Header 149

Header 150

Header 151

Header 152

Header 153

Header 154

Header 155

Header 156

Header 157

Header 158

Header 159

Header 160

Header 161

Header 162

Header 163

Header 164

Header 165

Header 166

Header 167

Header 168

Header 169

Header 170

Header 171

Header 172

Header 173

Header 174

Header 175

Header 176

Header 177

Header 178

Header 179

Header 180

Header 181

Header 182

Header 183

Header 184

Header 185

Header 186

Header 187

Header 188

Header 189

Header 190

Header 191

Header 192

Header 193

Header 194

Header 195

EXERCISE 3.1

Play around with the different chunk parameters until the first two chunks look like the ones in the image.

R-Course

Sofia Gil-Clavel

2026-07-23

Example code for R-Markdown usage

Preparing the environment

In the following chunk, I clean the *R environment*.

```
rm(list=ls())  
gc()
```

Then I load the packages.

```
library(tidyverse)
```

Performing Data Handling

For the *data handling*, first, I turn the database iris into a **tibble**.

```
IRIS<-tibble(iris)
```

Finally, I obtain the **mean** petal length by iris species.

```
IRIS2<-IRIS%>%  
  group_by(Species)%>%  
  summarise(MEAN=mean(Petal.Length))
```

```
IRIS2
```

```
## # A tibble: 3 x 2  
##   Species    MEAN  
##   <fct>    <dbl>  
## 1 setosa    1.46  
## 2 versicolor 4.26  
## 3 virginica 5.55
```

Performing Data Visualization

To visualize the *results*, I plot the values using bar plots.



CREATING TABLES

CREATING TABLES

```
```{r data handling}

IRIS2<-IRIS%>%
 group_by(Species)%>%
 summarise(MEAN=mean(Petal.Length))

```
```

```
```{r formating as table, echo=FALSE}

knitr::kable(IRIS2)

```
```

To create a table, we will use the function “kable()” from the package “knitr”.

You should already have “knitr” installed, as it is used to render the R-Markdown files.

Finally, I obtain the **mean** petal length by iris species.

```
IRIS2<-IRIS%>%
  group_by(Species)%>%
  summarise(MEAN=mean(Petal.Length))
```

| Species | MEAN |
|------------|-------|
| setosa | 1.462 |
| versicolor | 4.260 |
| virginica | 5.552 |

EXERCISE 3.2

Display a table with the minimum, maximum, mean, and variance of “Petal.Width” by Species.



FORMATTING GRAPHS

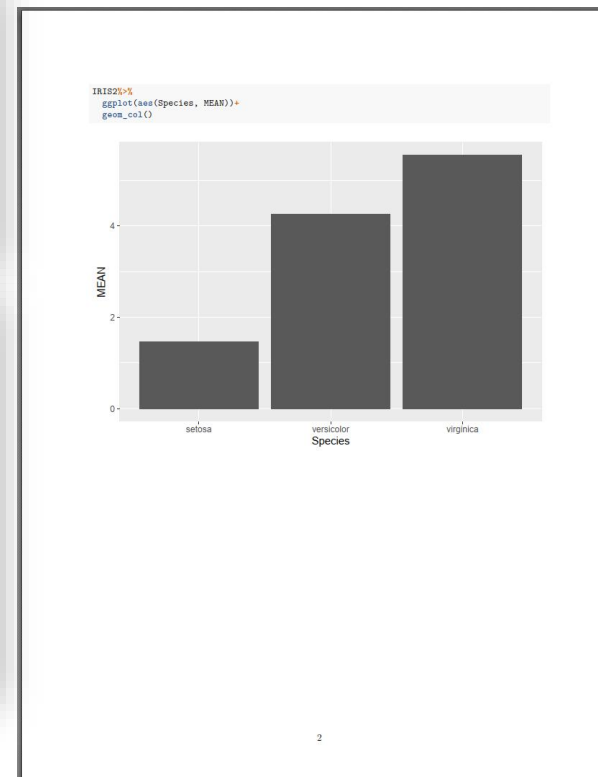
FORMATTING GRAPHS

Option 1: At the document level.

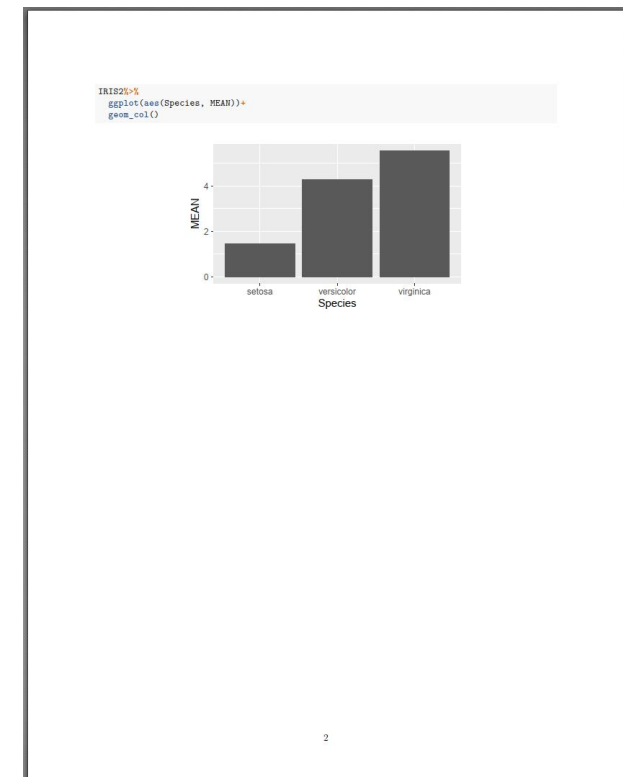
```
14
15 ```{r cleaning environment, results='hide'}
16
17 rm(list=ls())
18 gc()
19
20 ```
21
22 ```{r formatting the graph, echo=FALSE}
23
24 knitr::opts_chunk$set(
25   fig.align = 'center',
26   fig.height = 2.5, fig.width = 4,
27   fig.cap = NULL,
28   fig.path = "FIGURES/"
29 )
30
31 ```
```

| | | |
|-----------------------------------|-----------|---|
| fig.align | "default" | "left", "right", or "center" |
| fig.alt | NULL | alt text for a figure |
| fig.cap | NULL | figure caption as a character string |
| fig.path | "figure/" | prefix for generating figure file paths |
| fig.width & fig.height | 7 | plot dimensions in inches |

Before



After



FORMATTING GRAPHS

Option 2: At the chunk level, this gives more nuance control, i.e., it gives more parameters to play with.

```
70 ## Performing Data Visualization
71
72 To visualize the *results*, I plot the values using
73 bar plots.
74 ```{r visualizing results, fig}
75
76 IRIS2%>%
77   ggplot(aes(Species, MEAN))
78   geom_col()
79
80 ```
```

- ◆ fig.align=
- ◆ fig.alt=
- ◆ fig.asp=
- ◆ fig.cap=
- ◆ fig.dim=
- ◆ fig.dpi=
- ◆ fig.env=
- ◆ fig.ext=

74:31 Chunk 7: visualizing results R Markdown

75 ```{r visualizing results, fig.dpi=300}

```
76
77 IRIS2%>%
78   ggplot(aes(Species, MEAN))+
79   geom_col()
80
81 ```
```

EXERCISE 3.3

Display the box plots of “Petal.Width” by Species.

5 MINS BREAK



4. BUILDING ON AXIOMS

THE LOGIC OF MATH

In math **axioms** are the foundational “rules of the game”.

An axiom (or postulate) is a statement that is accepted as true without proof.

They serve as the starting points from which all other mathematical truths are logically derived.

The steps to derive knowledge from axioms:

1. **Start with Axioms:** You accept a set of core axioms (and any previous theorems already proven using those axioms).
2. **Apply Rules of Logic:** You use valid logical steps (like *if A implies B , and A is true, then B must be true*).
3. **Reach a Conclusion:** You arrive at a new statement, known as a **theorem**.
4. Once a theorem is proven, it can be added to your toolkit alongside the axioms to help prove even more complex theorems!

MATH LOGIC IN PRACTICE

Using the addition, multiplication, and connecting axioms prove that:

$$(a + b)^2 = a^2 + 2 \cdot a \cdot b + b^2$$

Proof:

$$\begin{aligned}(a + b)^2 &= (a + b) \cdot (a + b) \text{ by exponentiation definition.} \\ &= \underbrace{a \cdot (a + b)} + \underbrace{b \cdot (a + b)} \text{ by distributive axiom.} \\ &= a \cdot a + a \cdot b + \underbrace{b \cdot a} + b \cdot b \text{ by distributive axiom.} \\ &= a \cdot a + \underbrace{a \cdot b + a \cdot b} + b \cdot b \text{ by mult. commutative.} \\ &= \underbrace{a \cdot a} + 2 \cdot \underbrace{a \cdot b} + \underbrace{b \cdot b} \text{ by addition definition.} \\ &= \underbrace{a^2} + 2 \cdot a \cdot b + \underbrace{b^2} \text{ by exponentiation definition.}\end{aligned}$$

Addition Axioms

- **Commutative:** $a + b = b + a$ (order does not change the sum).
- **Associative:** $(a + b) + c = a + (b + c)$ (grouping does not change the sum).
- **Identity:** $a + 0 = a$ (adding zero leaves the number unchanged).
- **Inverse:** $a + (-a) = 0$ (every number has an opposite that yields zero).

Multiplication Axioms

- **Commutative:** $a \cdot b = b \cdot a$ (order does not change the product).
- **Associative:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ (grouping does not change the product).
- **Identity:** $a \cdot 1 = a$ (multiplying by one leaves the number unchanged).
- **Inverse:** $a \cdot a^{-1} = 1$ for any $a \neq 0$ (every non-zero number has a reciprocal).

Connecting Axiom

- **Distributive:** $a \cdot (b + c) = a \cdot b + a \cdot c$ (multiplication spreads over addition).

MATH LOGIC IN CODING

We can think of functions as axioms and packages as a set of axioms that will serve as foundation to build other code.

The assumption is that the functions in those packages are correct and that if we apply them logically, then the results should also be correct.

The steps to derive knowledge from packages:

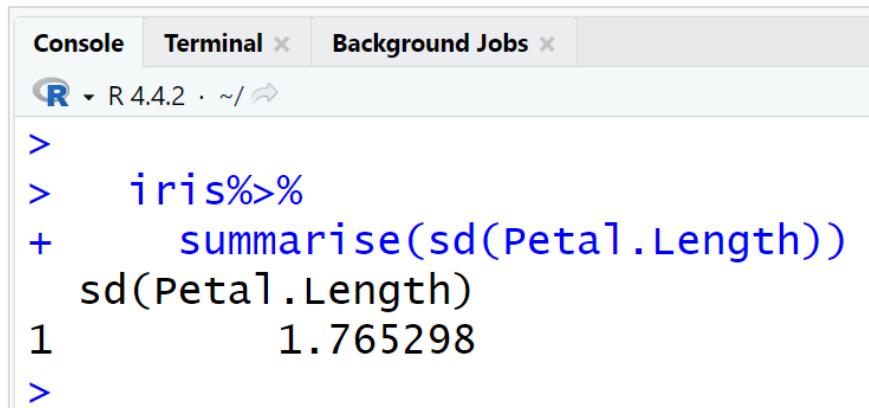
1. **Start with a package:** You accept a set of core functions (and any previous functions used to build those functions).
2. **Apply Rules of Logic:** You use valid logical steps (like *if A implies B, and A is true, then B must be true*).
3. **Reach a Conclusion:** You arrive at a new statement, known as a **new function or result**.
4. Once a function is proven, it can be added to your toolkit alongside the packages that helped build even more complex functions and code!

CODING WITH MATH LOGIC

Using the packages base R, pipe, and dplyr prove that the standard deviation of Petal.Length is 1.77.

```
library(dplyr)

iris%>%
  summarise(sd(Petal.Length))
```



The screenshot shows an R console window with tabs for 'Console', 'Terminal', and 'Background Jobs'. The console output shows the R prompt, the library loading, the pipe operation, and the final standard deviation result.

```
>
> library(dplyr)
> iris%>%
+   summarise(sd(Petal.Length))
1      1.765298
>
```

Base R

Standard Deviation

Description

This function computes the standard deviation of the values in `x`. If `na.rm` is `TRUE` then missing values are removed before computation proceeds.

Usage

```
sd(x, na.rm = FALSE)
```

Pipe

Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

Here, “equivalent” is not technically exact: evaluation is non-standard, and the left-hand side is evaluated before passed on to the right-hand side expression. However, in most cases this has no practical implication.

Dplyr

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

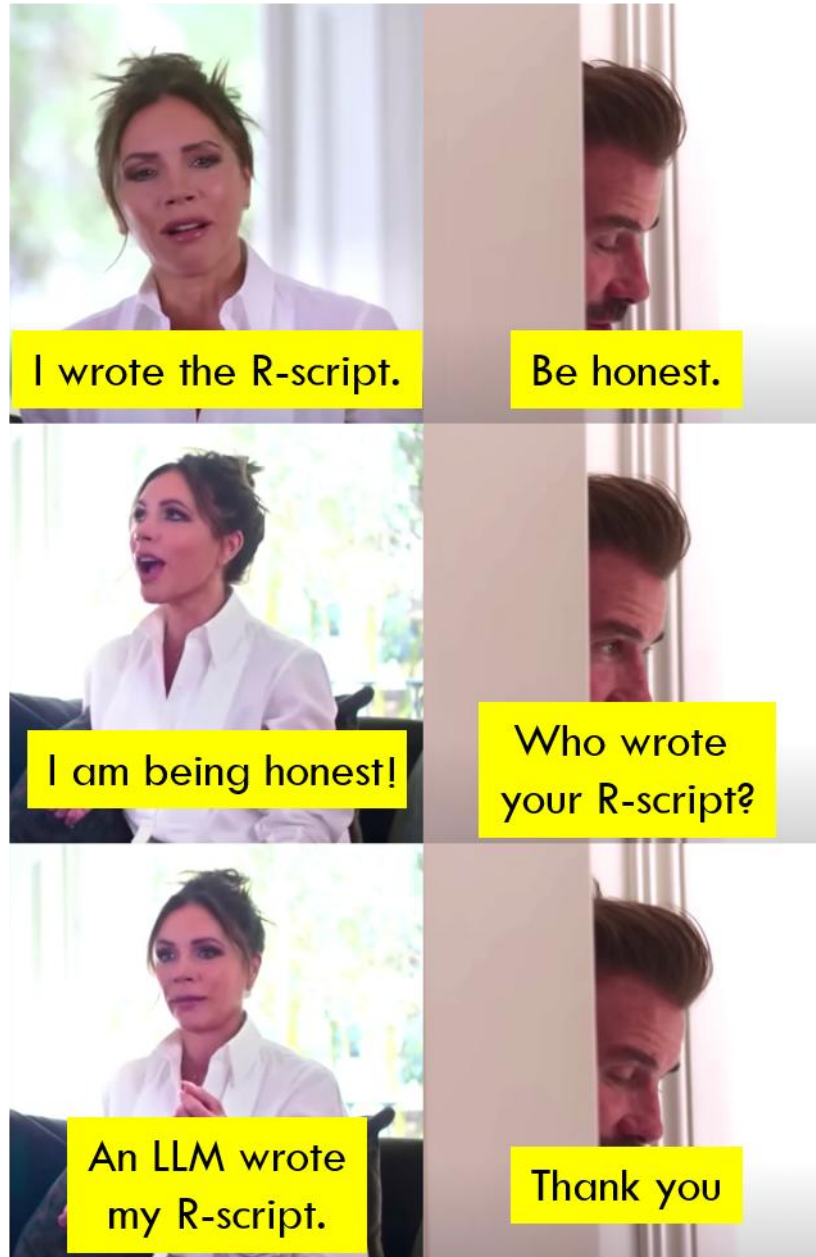
EXERCISE 4.1

Using the mean function and the tidyverse package show that on average the petal length of the Virginica species of an Iris flower is 3.8 times bigger than the Setosa species. Explain the logic of each code step.

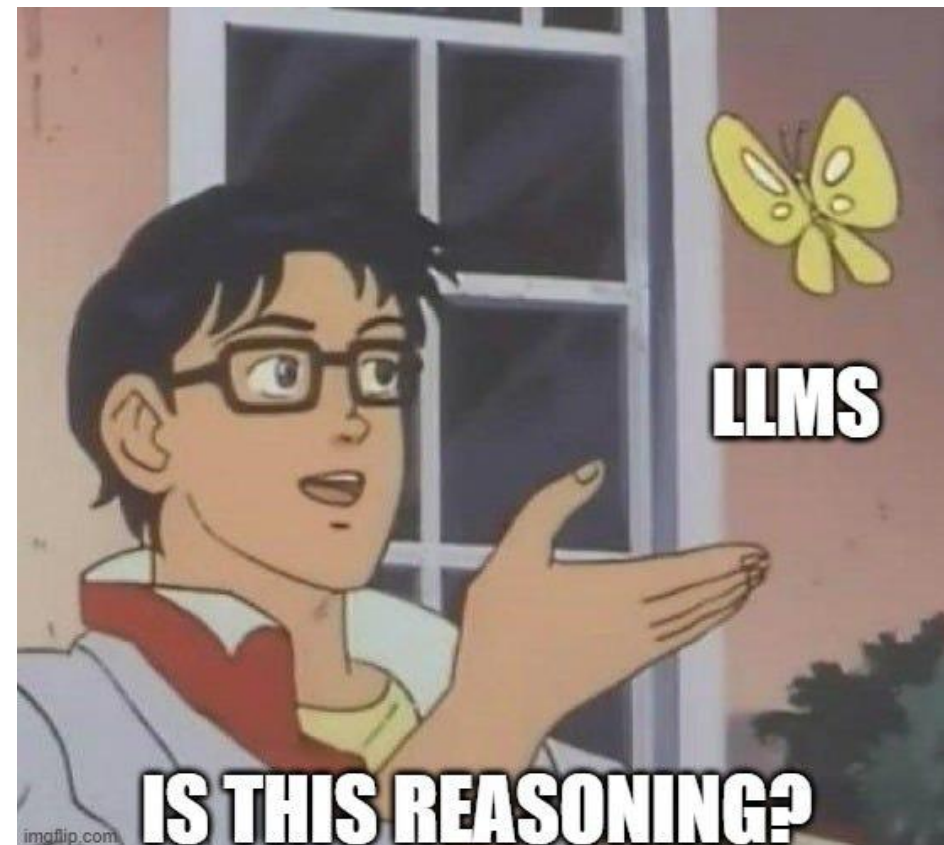
5. TRACKING STUDENTS' LOGIC

Student

Teacher

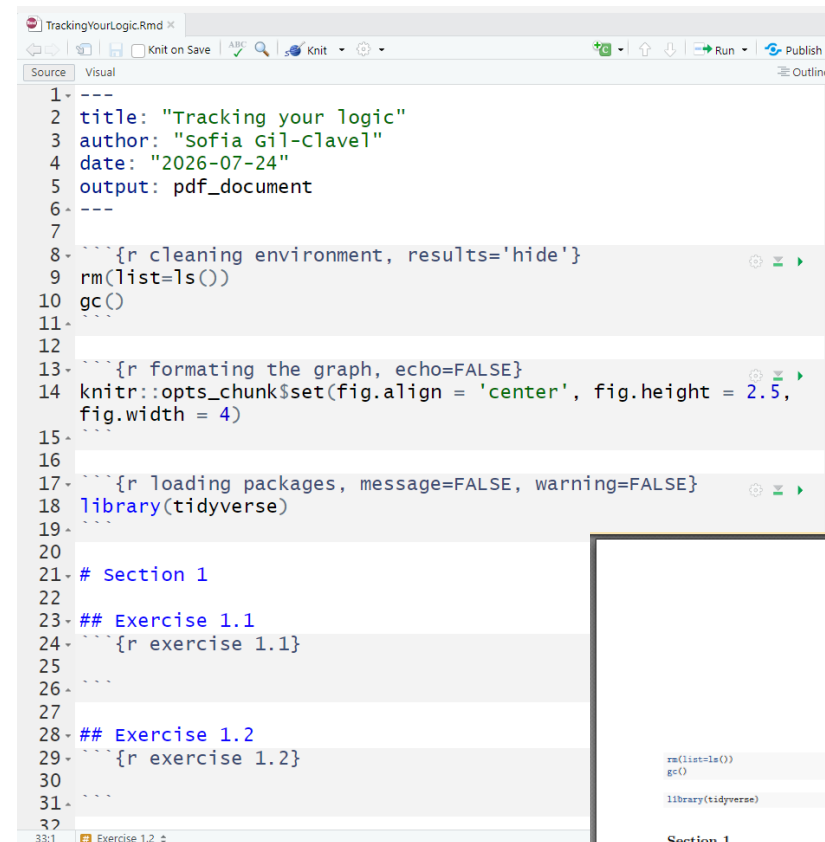


THE NEW CHALLENGE!

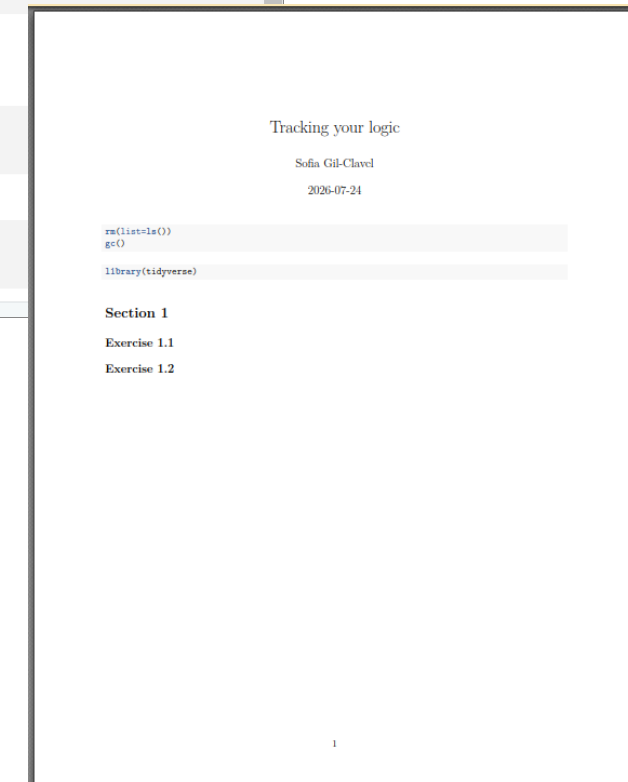


R-MARKDOWN ALLOWS TRACKING STUDENTS' LOGIC

1. Create an R-Markdown.
2. In the first chunk clean the environment.
3. In the second chunk open the packages that are allowed for the assignment.
4. In the subsequent sections create the chunks where students must write their answers.
5. Request students to submit both the R-Markdown with their answers and the result of rendering the pdf.
6. Remember that the document doesn't render when the code doesn't work.



```
1 ---
2 title: "Tracking your logic"
3 author: "Sofia Gil-Clavel"
4 date: "2026-07-24"
5 output: pdf_document
6 ---
7
8 {r cleaning environment, results='hide'}
9 rm(list=ls())
10 gc()
11
12
13 {r formatting the graph, echo=FALSE}
14 knitr::opts_chunk$set(fig.align = 'center', fig.height = 2.5,
15   fig.width = 4)
16
17 {r loading packages, message=FALSE, warning=FALSE}
18 library(tidyverse)
19
20
21 # Section 1
22
23 ## Exercise 1.1
24 {r exercise 1.1}
25
26
27
28 ## Exercise 1.2
29 {r exercise 1.2}
30
31
32
33 Exercise 1.2
```



EXERCISE 5.1

1. Answer the exercises in the R-Markdown “Tracking_Your_Logic.Rmd”.
2. Render the file.

EXERCISE 5.2

1. Compare your code and results with the person to your right, or with the participants online.
2. Did you all arrive to the same results?
3. Did you all code the same?

REFERENCES

- ❑ Xie, Yihui, J. J. Allaire, and Garrett Grolemond. “R Markdown: The Definitive Guide.” Routledge & CRC Press. Accessed July 21, 2026.
<https://www.routledge.com/R-Markdown-The-Definitive-Guide/Xie-Allaire-Grolemond/p/book/9781138359338>. It can be accessed for free here:
<https://pkg.yihui.org/rmarkdown-book/>



Join us!



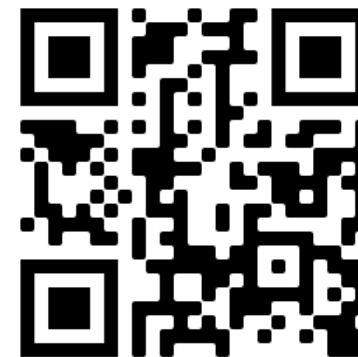
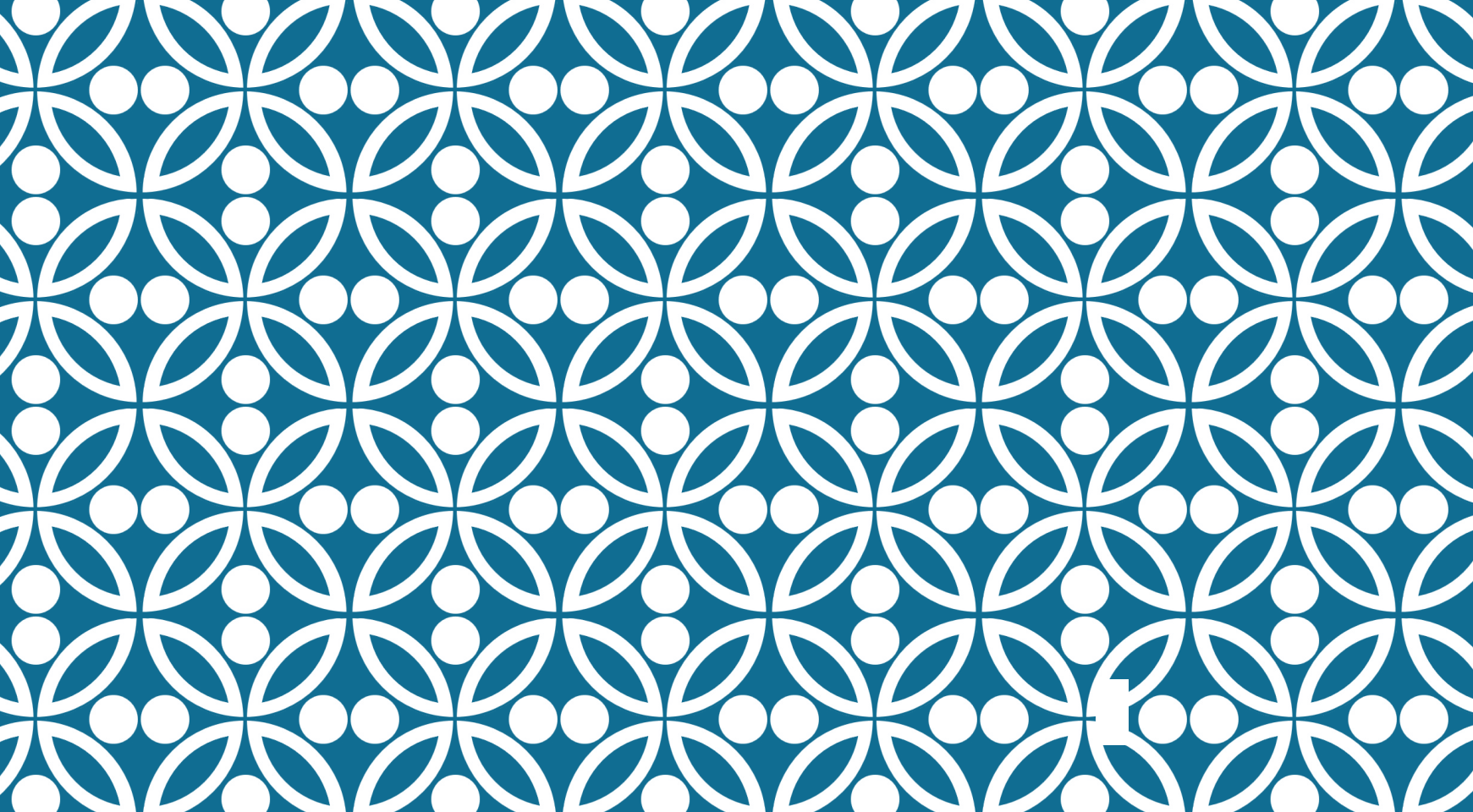
<https://forms.office.com/e/8Bgd2YsasJ>

All about the lab:

<https://societal-analytics.nl/>

Contact us at:

analytics-lab.fsw@vu.nl



<https://sofiag1l.github.io/>

THANKS!

Dr. Sofia Gil-Clavel